GB3 RISC-V: Processor Architecture and Optimizations

1 Introduction

This document is a technical reference for the GB3 RISC-V project, covering the iCE40UP5K FPGA platform, the picorv32 processor and PicoSoC, and background on the design of microarchitecture. It condenses the core content of Harris [2] (Chapters 2–5) and Patterson [1] alongside analysis of the picorv32 microarchitecture.

2 Processor Architecture

A processor executes a program by repeatedly performing the same loop: **fetch** an instruction from memory, **decode** it to determine what operation to perform, **execute** the operation, and **write back** the result. Understanding a processor requires separating two levels of abstraction:

- The **instruction set architecture (ISA)** is the programmer's contract with the hardware. It defines the set of instructions the processor understands *(instruction set)*, the registers available, how memory is addressed, and how instructions are encoded as binary *(instruction format)*. Multiple chips can implement the same ISA.
- The **microarchitecture** is the hardware implementation of the ISA. It determines *how* each instruction is executed: how many clock cycles it takes, how the datapath is organised (ALU, register file, memory interface), and how the control logic sequences the operations. The same ISA can be implemented as a single-cycle, multicycle, or pipelined microarchitecture, each with different trade-offs in speed, area, and power.

RISC-V is an ISA, and the picorv32 is one microarchitecture that implements it. The next few sections condenses the Harris [2] textbook, where Section 3 covers the ISA (Harris Chapter 6); Section 4 covers the microarchitecture (Harris Chapter 7) and Section 5 covers memory systems (Harris Chapter 8).

3 RV32I Instruction Set Architecture

RISC-V ISA is motivated by four design principles articulated by Patterson and Hennessy:

1. **Regularity supports simplicity.** A consistent instruction format — same number of operands, same field positions — simplifies the decoder hardware. This is why every RV32I instruction is 32 bits, `opcode` is always at [6:0], `rs1` always at [19:15], and so on.
2. **Make the common case fast.** Keep the instruction set small so that each instruction can be decoded and executed quickly. Complex operations (e.g. string copy) are built from sequences of simple instructions, rather than adding slow, seldom-used complex instructions.
3. **Smaller is faster.** Fewer registers mean shorter addresses and faster access. 32 registers is a sweet spot: enough to avoid excessive spilling to memory, few enough that each register address fits in 5 bits.
4. **Good design demands good compromises.** A single instruction format would be simplest but too restrictive; many formats would be flexible but hard to decode. RISC-V compromises with four main formats (R, I, S/B, U/J), balancing regularity with expressiveness.

For reference tables of RISC-V compressed instructions, register names, instruction formats etc refer to the first few pages of [2].

3.1 Assembly Language and Machine Language

A programmer might write instructions in **assembly language**, a human-readable notation where each line specifies an operation (e.g. `add`) and its operands (usually data stored in registers, memory, or explicitly encoded in the instruction itself):

```
add s2, s3, s4     (add contents of registers s3 and s4, store result in s2)
```

The assembler translates each instruction into **machine language** — the 32-bit binary word the hardware actually fetches and decodes. Each field in the binary word encodes a specific piece of information:

`funct7`	`rs2`	`rs1`	`funct3`	`rd`	`opcode`	
`0000000`	`10100`	`10011`	`000`	`10010`	`0110011`	= `0x01498933`
7 bits	s4 (x20)	s3 (x19)	add	s2 (x18)	R-type ALU	

The processor never sees the text "`add s2, s3, s4`" — it sees only the 32-bit number `0x01498933` fetched from instruction memory. The structure of these 32-bit words is defined by the *instruction formats*.

3.2 RISC-V Registers

The RISC-V ISA defines a **32-bit architectural state**: **32 general-purpose registers** `x0`–`x31` and a program counter (PC). Register `x0` is hardwired to zero; writes to it are discarded. This constraint simplifies the ISA — pseudo-instructions like `nop` (= `addi x0, x0, 0`) and `mv` (= `addi rd, rs1, 0`) fall out naturally.

Name	Reg	Use	Preserved?
`zero`	`x0`	Constant 0	—
`ra`	`x1`	Return address	Callee
`sp`	`x2`	Stack pointer	Callee
`gp`	`x3`	Global pointer	—
`tp`	`x4`	Thread pointer	—
`t0–t2`	`x5–x7`	Temporaries	Caller
`s0/fp`	`x8`	Saved / frame pointer	Callee
`s1`	`x9`	Saved	Callee
`a0–a1`	`x10–x11`	Args / return values	Caller
`a2–a7`	`x12–x17`	Arguments	Caller
`s2–s11`	`x18–x27`	Saved	Callee
`t3–t6`	`x28–x31`	Temporaries	Caller

Table 1: RV32I register naming conventions and usage. "Callee" = must be saved/restored by called function; "Caller" = may be clobbered by any call.

3.3 RISC-V Memory model and Addressing modes

To address the data stored in memory, RISC-V uses a **byte-addressable** (each byte is mapped to unique address), **little-endian** (address is LSB) memory space with 32-bit addresses ($2^{32} = 4$ GB). Each 32 bit word consists of four 8-bit bytes, so each word address also has 32 bits, and are multiples of 4. All data transfer between registers and memory goes through explicit load/store instructions — there are no memory operands in ALU instructions (this is the defining feature of a *load-store architecture*). Instructions can only operate on registers, not directly on memory. Figure 1 shows how bytes and words are arranged. Each word occupies 4 consecutive byte addresses; the word address is the address of its lowest byte [1]

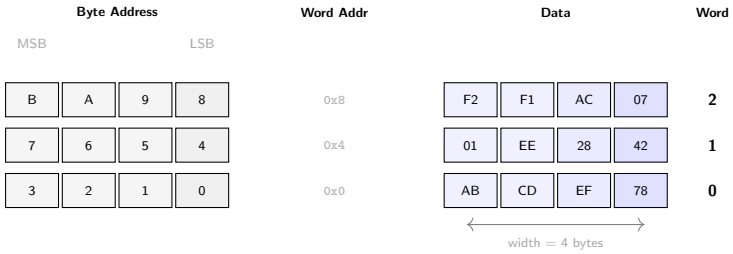


Figure 1: Byte-addressable memory (little-endian), after Harris Figure 6.1. Left: byte addresses within each word. Right: example data values. The word address equals the byte address of the LSB (lowest address). E.g. Word `0xF2F1AC07` is stored at address `0x4`. `lw` reads all 4 bytes; `lh` reads 2 bytes; `lb` reads one byte.

Load instructions read from memory into a register: `lw rd, offset(rs1)` computes address = `rs1` + sign-extend(offset) and writes the 32-bit word at that address into `rd`. Sub-word loads `lh`/`lb` sign-extend; `lhu`/`lbu` zero-extend. **Store** instructions write a register to memory: `sw rs2, offset(rs1)` writes `rs2` to memory at `rs1` + offset.

RISC-V uses four addressing modes:

Mode	Used by	Address computation
Register	R-type ALU	Operand is a register value
Immediate	I-type ALU	Operand is a sign-extended 12-bit constant
Base	Loads / stores	`rs1` + imm_{12} (base + offset)
PC-relative	Branches, `jal`, `auipc`	`PC` + imm

The base addressing mode is what drives all memory access: the ALU computes `reg_op1 + decoded_imm` to produce the memory address. For memory-mapped I/O (Section 7.7.3), the same `lw`/`sw` instructions access peripheral registers — the address decoder routes the transaction to the correct device.

3.4 Programming Model

3.4.1 Program flow and the PC

Instructions are stored in memory as 32-bit words (4 bytes), so consecutive instruction addresses increase by four. The **program counter (PC)** holds the address of the current instruction. After each instruction completes, the PC increments by 4 to fetch the next:

Address	Instruction
`0x538`	`addi s1, s2, s3`
`0x53C`	`lw  t2, 8(s1)`
`0x540`	`sw  s3, 3(t6)`

[1] RISC-V is little-endian, so the least significant byte is at the lowest address

When the processor executes the `addi` at `0x538`, PC = `0x538`. After completion, PC increments to `0x53C` and the processor fetches `lw`. This sequential flow continues unless a branch or jump redirects the PC.

3.4.2 Logical, shift, and multiply instructions

Beyond arithmetic (`add`/`sub`), RV32I provides bitwise logical operations and shifts, all available in both register–register (R-type) and register–immediate (I-type) forms:

Operation	R-type	I-type
AND / OR / XOR	`and rd, rs1, rs2`	`andi rd, rs1, imm`
Shift left logical	`sll rd, rs1, rs2`	`slli rd, rs1, shamt`
Shift right logical	`srl rd, rs1, rs2`	`srli rd, rs1, shamt`
Shift right arithmetic	`sra rd, rs1, rs2`	`srai rd, rs1, shamt`

Shifts are particularly important for array indexing: `slli rd, rs1, 2` multiplies by 4 to convert a word index to a byte offset. The M extension adds `mul`/`div`/`rem` (Section 7.4), but the base ISA handles multiplication by powers of 2 via shifts.

3.4.3 Branching and conditional statements

RV32I provides six conditional branch instructions. All are B-type (two register sources, a 13-bit PC-relative offset):

Instruction	Branches if
`beq rs1, rs2, label`	rs1 $=$ rs2
`bne rs1, rs2, label`	rs1 $\neq$ rs2
`blt / bltu`	rs1 $<$ rs2 (signed / unsigned)
`bge / bgeu`	rs1 $\geq$ rs2 (signed / unsigned)

The key assembly pattern: test the *opposite* condition to *skip* the block. An `if (a==b)` in C becomes `bne a, b, skip` — if not equal, skip the if-body. For `if/else`, the if-body ends with a `j` (unconditional jump) past the else-body:

```
    # if (s0 == s1) { s2 = s3 + s4; } else { s0 = s1 - s4; }
    bne  s0, s1, else     # skip to else if s0 != s1
    add  s2, s3, s4       # if-body
    j    done
else:
    sub  s0, s1, s4       # else-body
done:
```

`switch/case` compiles to a chain of compare-and-branch pairs (load constant into a temporary, `bne` to the next case), equivalent to nested `if/else`.

3.4.4 Loops

All loop forms use branches. A **while** loop tests the (opposite) condition at the top and jumps back from the bottom:

```
    # while (s0 != 128) { s0 = s0 * 2; s1++; }
    addi t0, zero, 128
while: beq  s0, t0, done   # exit if s0 == 128
    slli s0, s0, 1          # s0 *= 2
    addi s1, s1, 1
    j    while
done:
```

A **do/while** is identical but places the branch at the bottom (test the *same* condition to continue, no initial jump). A **for** loop is syntactic sugar over while: initialisation before the loop, increment at the bottom, same opposite-condition test at the top.

3.4.5 Arrays

An array stores elements (bytes) at sequential addresses. For 32-bit integers, element i is at address base $+ 4i$. The standard access pattern multiplies the index by 4 (shift left by 2), adds the base, and uses `lw`/`sw`: [2]

```
    # scores[i] += 10,  s0 = base address of scores,  s1 = index i
    slli t0, s1, 2        # t0 = i << 2 = i * 4  (byte offset for word-sized elements)
    add  t0, t0, s0       # t0 = base + 4i = &scores[i]  (address of element i)
    lw   t1, 0(t0)        # t1 = MEM[base + 4i] = scores[i]  (load element)
    addi t1, t1, 10       # t1 = scores[i] + 10
    sw   t1, 0(t0)        # MEM[base + 4i] = t1  (store back to scores[i])
```

3.4.6 Function calls and the stack

High-level languages use **functions** (also called procedures or subroutines) to reuse common code and make programs more modular and readable. Functions may have inputs, called *arguments*, and an output, called the *return value*. A well-behaved function should calculate its return value and cause no other unintended side effects. When one function calls another, the calling function (the **caller**) and the called function (the **callee**) must agree on several

[2] The syntax `offset(base)` denotes the memory address `base + offset`, where `base` is a register and `offset` is a 12-bit signed immediate. For example, `lw t1, 0(t0)` loads from address `t0 + 0`, and `sw s3, 8(sp)` stores to address `sp + 8`.

things: where to put the arguments, where to find the return value, and where the callee should return to when it finishes. In RISC-V, the calling convention specifies:

- The caller places up to 8 arguments in `a0`–`a7` before making the call; additional arguments go on the stack.
- The caller stores the return address in `ra` at the same time it jumps to the callee, using `jal`.
- The callee places the return value in `a0` (and `a1` for 64-bit results) before returning.
- The callee must not overwrite any state the caller depends on. Specifically, it must leave the *saved registers* (`s0`–`s11`), the return address (`ra`), the stack pointer (`sp`), and all memory above `sp` unmodified.

Call and return. Two instructions handle function calls:

- `jal rd, target` (*jump and link*): jumps to the PC-relative target and simultaneously stores the return address (`PC+4`) in `rd`. By convention `rd` = `ra` (x1), so `jal func` is shorthand for `jal ra, func`.
- `jalr rd, rs1, imm` (*jump and link register*): jumps to the address `rs1+imm` and stores `PC+4` in `rd`. The pseudoinstruction `ret` ≡ `jalr x0, ra, 0` returns by jumping to the address saved in `ra`.

A minimal example (Harris Code Example 6.22): `main` calls `simple`, which immediately returns:

```
0x300 main: jal simple    # ra = 0x304, jump to 0x51C
0x304 ...                 # execution resumes here after return
...
0x51C simple: jr ra       # jump to address in ra (0x304)
```

`jal simple` does two things at once: it jumps to the target instruction at `0x51C` and stores the return address (`0x304`, the instruction after `jal`) in `ra`. The callee returns by executing `jr ra` (= `jalr x0, ra, 0`), which jumps back to `0x304`. The `main` function then continues executing at this address.

Arguments and return values. The `simple` function above is not very useful — it receives no input and returns no output. A more realistic function, `diffofsums`, takes four arguments and returns one result. The caller places the arguments in `a0`–`a3`; the callee puts the return value in `a0`:

```
# y = diffofsums(f, g, h, i) = (f+g)-(h+i)
# y = diffofsums(2, 3, 4, 5)    -- caller (main)
addi a0, zero, 2        # argument 0 = 2
addi a1, zero, 3        # argument 1 = 3
addi a2, zero, 4        # argument 2 = 4
addi a3, zero, 5        # argument 3 = 5
jal  diffofsums          # call function
add  s7, a0, zero        # y = returned value

diffofsums:              # -- callee
add  t0, a0, a1          # t0 = f + g
add  t1, a2, a3          # t1 = h + i
sub  s3, t0, t1          # result = (f+g) - (h+i)
add  a0, s3, zero        # put return value in a0
jr   ra                  # return to caller
```

But this version of `diffofsums` has a problem: it modifies `t0`, `t1`, and `s3`. If `main` was using any of those registers before the call, their contents would be silently corrupted. To solve this, a function must save register values on the stack before modifying them and restore them before returning.

The stack. The stack is scratch memory for saving registers and storing local variables. It is a last-in-first-out (LIFO) structure: the last word pushed is the first one popped. RISC-V uses a **full descending stack**: the stack pointer `sp` (register `x2`) starts at a high address and is decremented to grow (allocate) and incremented to shrink (deallocate). The stack space a function allocates for itself is its **stack frame**.

A function that needs to save registers follows a five-step pattern:

1. Decrement `sp` to make room on the stack
2. Store (`sw`) the register values into the allocated space
3. Execute the function body, freely using those registers
4. Load (`lw`) the original values back from the stack
5. Increment `sp` to deallocate the space

Hence the stack is a temporary location to preserve the contents of registers so operations are free to read and write to registers, and then load back values when done. Code Example 6.24 shows the improved `diffofsums` that saves and restores all three registers it modifies:

```
diffofsums:
addi sp, sp, -12         # 1. allocate 3 words of space on stack
sw   s3, 8(sp)           # 2. save s3, to sp+8:sp+12
sw   t0, 4(sp)           #    save t0, to sp+4:sp+8
sw   t1, 0(sp)           #    save t1, to sp:sp+4
add  t0, a0, a1          # 3. now can use registers freely. t0 = f + g
add  t1, a2, a3          #    t1 = h + i
sub  s3, t0, t1          #    result = (f+g) - (h+i)
add  a0, s3, zero        #    return value in a0
lw   s3, 8(sp)           # 4. restore s3
lw   t0, 4(sp)           #    restore t0
lw   t1, 0(sp)           #    restore t1
addi sp, sp, 12          # 5. deallocate
jr   ra                  #    return
```

Figure 2 shows the stack before, during, and after this call. The key invariant: `sp` and all memory above it are identical before and after execution.

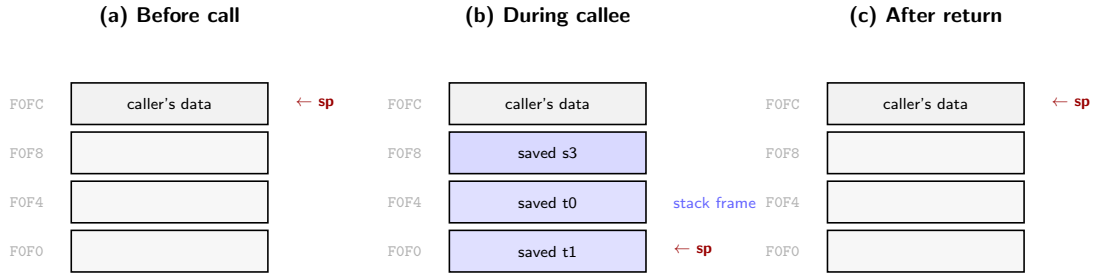


Figure 2: Stack before, during, and after a function call (after Harris Figure 6.8). The callee decrements `sp` by 12 to allocate a 3-word frame, saves `s3`, `t0`, and `t1`, executes, restores them, and increments `sp`. The stack is identical in (a) and (c) — no side effects beyond `a0`.

Preserved vs. nonpreserved registers. However, saving all three registers is wasteful. If the caller was not using `t0` and `t1`, the effort to save and restore them is wasted. To avoid this waste, RISC-V divides registers into **preserved** (`s0`–`s11`, `ra`, `sp` — callee must save value to stack before modifying the register) and **nonpreserved** (`t0`–`t6`, `a0`–`a7` — callee may freely modify register contents). Since `t0` and `t1` are nonpreserved, only `s3` actually needs saving. A **leaf function** (one that calls no others) can avoid the stack entirely by using only nonpreserved registers. A **nonleaf function** (one that calls others) must always save `ra`, since `jal` overwrites it. Recursive functions are nonleaf functions that call themselves; each invocation creates a new stack frame, so the stack grows by one frame per recursion level.

3.4.7 Pseudoinstructions

RISC-V keeps the hardware instruction set small; the assembler provides pseudoinstructions that expand into one or two real instructions:

Pseudo	Expands to	Effect
`mv rd, rs`	`addi rd, rs, 0`	Copy register
`li rd, imm`	`lui + addi` (or just `addi`)	Load constant
`j label`	`jal x0, label`	Unconditional jump (no link)
`ret`	`jalr x0, ra, 0`	Return from function
`call label`	`auipc ra, ...  + jalr ra, ra, ...`	Far function call
`nop`	`addi x0, x0, 0`	No operation
`not rd, rs`	`xori rd, rs, -1`	Bitwise NOT

3.5 Machine Language

3.5.1 Instruction formats

Every RV32I instruction is exactly 32 bits. The ISA defines six formats, shown in Figure 3.

- **Opcode** (`instr[6:0]`) is in the same position in every format. This lets the decoder determine the format with a single 7-bit lookup before parsing the rest of the word.
- **rd** (destination register, `[11:7]`), **rs1**/**rs2** (source registers, `[19:15]` and `[24:20]`) are always at fixed positions whenever they exist. Each register address is 5 bits exactly. The register file can begin reading before the full decode is complete.
- **funct3** (`[14:12]`) distinguishes operations within the same opcode class (e.g. `add` vs `and` vs `slt` all share the R-type opcode). **funct7** (`[31:25]`) provides further disambiguation in R-type: bit 5 separates `add` from `sub` and `srl` from `sra`.
- **The sign bit** of every immediate is always `instr[31]`.

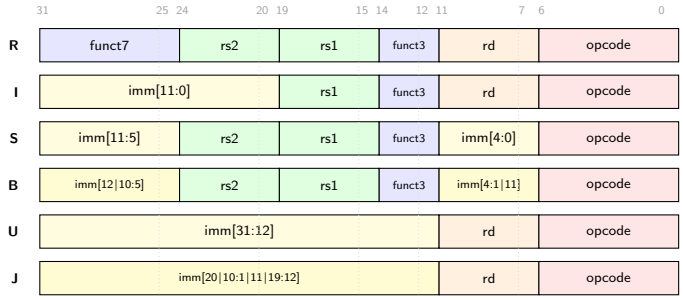


Figure 3: RV32I instruction formats. Each column corresponds to a fixed bit position: `opcode` (red) is always [6:0], `rs1` (green) always [19:15], `rs2` always [24:20], `funct3` (blue) always [14:12]. Yellow fields are immediates (sign bit always at bit 31). Dotted vertical lines highlight the alignment.

Table 2 lists the instruction classes with their operations and opcodes.

Table 3 expands each class into its individual operations, showing how `funct3` (and where applicable `funct7`) selects the specific instruction within each opcode group.

Class	Fmt	opcode	Operation	Description
R-type ALU	R	0110011	`rd = rs1 op rs2`	Register–register arithmetic/logic. The `funct3` field selects the operation (add, sub, and, or, xor, sll, srl/sra, slt/sltu); `funct7` bit 5 distinguishes add/sub and srl/sra.
I-type ALU	I	0010011	`rd = rs1 op imm`	Register–immediate: same operations as R-type, but the second operand is a 12-bit sign-extended constant from the instruction word. Shift immediates use only bits [4:0] as the shift amount.
Loads	I	0000011	`rd = Mem[rs1+imm]`	Reads from memory at address `rs1 + sign_ext(imm)`. `lw` loads a full 32-bit word; `lh`/`lb` load 16/8 bits and sign-extend to 32; `lhu`/`lbu` zero-extend instead.
Stores	S	0100011	`Mem[rs1+imm] = rs2`	Writes `rs2` to memory at address `rs1 + sign_ext(imm)`. No destination register — nothing is written to the register file. The immediate is split across two fields (`[31:25]` and `[11:7]`).
Branches	B	1100011	`if(rs1 cmp rs2) PC+=imm`	Conditional PC-relative branch. Compares two registers and, if true, adds a 13-bit signed offset (LSB always 0) to PC. Six comparisons via `funct3`: eq, ne, lt, ge, ltu, geu.
`jal`	J	1101111	`rd=PC+4; PC+=imm`	Unconditional jump with link. Saves the return address (PC+4) in `rd` and adds a 21-bit signed offset to PC. Used for function calls (`rd=ra`) and unconditional jumps (`rd=x0`).
`jalr`	I	1100111	`rd=PC+4; PC=rs1+imm`	Jump to a register-computed address (`rs1+imm`), saving the return address in `rd`. Used for function returns (`ret` ≡ `jalr x0, ra, 0`), indirect calls, and computed jumps (switch tables).
`lui`	U	0110111	`rd = imm << 12`	Loads a 20-bit immediate into the upper 20 bits of `rd`, zeroing the lower 12. Paired with `addi` to build arbitrary 32-bit constants (`li` pseudoinstruction).
`auipc`	U	0010111	`rd = PC+(imm<<12)`	Like `lui` but adds the shifted immediate to the current PC. Enables position-independent code: paired with `jalr` for far function calls, or with `lw`/`sw` for PC-relative data access.

Table 2: RV32I instruction classes. The Operation column shows the register-transfer semantics; the Description column explains encoding details and typical usage.

Figure 4 shows which hardware components each instruction class uses. Every instruction reads the PC and instruction memory (fetch). The differences are in what happens next: ALU instructions use the register file and ALU but skip data memory; loads and stores use all four; branches use the ALU for comparison and conditionally update the PC; jumps write PC+4 to a register and update the PC unconditionally.



Figure 4: Datapath usage by instruction class. Filled dots show which stages are active; hollow dots show stages that are skipped. Every instruction uses the PC and instruction memory (fetch). ALU instructions skip data memory; stores skip writeback; branches conditionally update the PC.

3.5.2 Immediate encoding

Immediates are shorter than 32 bits (12 bits for I-type, 13 for B-type, 21 for J-type), but the ALU operates on 32-bit values. **Sign extension** pads the immediate to 32 bits by replicating its most significant (sign) bit into all the upper positions. For example, the 12-bit value `0xFFF` ($= -1$ in two's complement) becomes `0xFFFFFFFF` (still -1 as a 32-bit number), while `0x001` becomes `0x00000001`. This preserves the signed value of the immediate regardless of its original width.[3]

Shift immediates. The shift instructions `slli`/`srli`/`srai` use I-type encoding but only need 5 bits for the shift amount (`shamt` = `instr[24:20]`). The upper bits (`instr[31:25]`) are repurposed as a function code: `0000000` for `slli`/`srli`, `0100000` for `srai`.

The immediate bits are deliberately arranged for regularity across formats, so most of the immediate-extraction hardware is just wiring — only a small mux controlled by the opcode is needed for the roving bits. Because the sign bit is always at `instr[31]`, a single wire drives the sign-extension logic for all formats (see Harris Figures 6.26–6.27).

3.5.3 The stored program

A program in machine language is a series of 32-bit numbers stored in memory. The processor fetches them sequentially, decodes each one, executes it, and advances the PC by 4 to the next. This is the **stored program** concept:

[3] Since I-type immediates are only 12 bits, loading a full 32-bit constant requires two instructions: `lui` (upper 20 bits) + `addi` (lower 12); the `li` pseudoinstruction handles this.

Class	Fmt	Opcode	f3	Instr	Operation
R-type ALU	R	0110011	000	add/sub	rd = rs1 + rs2 or rs1 − rs2. Distinguished by funct7 bit 5 (0=add, 1=sub). The only R-type pair that uses funct7.
			001	sll	rd = rs1 << rs2[4:0]. Logical shift left; vacated bits filled with 0.
			010	slt	rd = (rs1 $<_s$ rs2) ? 1 : 0. Signed comparison; result is 0 or 1.
			011	sltu	rd = (rs1 $<_u$ rs2) ? 1 : 0. Unsigned comparison. sltu rd, x0, rs2 sets rd=1 if rs2≠0.
			100	xor	rd = rs1 ^ rs2. Bitwise exclusive OR.
			101	srl/sra	rd = rs1 >> rs2[4:0]. Logical or arithmetic; distinguished by funct7 bit 5.
			110	or	rd = rs1 | rs2. Bitwise OR.
			111	and	rd = rs1 & rs2. Bitwise AND.
I-type ALU	I	0010011	000	addi	rd = rs1 + sign_ext(imm). Most common instruction; mv, nop, li (small) are all addi variants.
			010	slti	rd = (rs1 $<_s$ imm) ? 1 : 0. Signed compare with immediate.
			011	sltiu	rd = (rs1 $<_u$ imm) ? 1 : 0. Unsigned compare. sltiu rd, rs1, 1 sets rd=1 if rs1=0.
			100	xori	rd = rs1 ^ sign_ext(imm). xori rd, rs1, -1 is the not pseudoinstruction.
			110	ori	rd = rs1 | sign_ext(imm). Bitwise OR with immediate.
			111	andi	rd = rs1 & sign_ext(imm). Bitwise AND with immediate; useful for masking bits.
			001	slli	rd = rs1 << shamt. Shift left by constant; shamt = imm[4:0], upper bits must be 0.
			101	srli/srai	rd = rs1 >> shamt. Logical or arithmetic; distinguished by imm[10] (0=srli, 1=srai).
Loads	I	0000011	000	lb	Load byte: rd = sign_ext(Mem[rs1+imm][7:0]). Reads 8 bits, sign-extends to 32.
			001	lh	Load halfword: rd = sign_ext(Mem[rs1+imm][15:0]). Reads 16 bits, sign-extends.
			010	lw	Load word: rd = Mem[rs1+imm]. Reads full 32 bits. Address must be 4-byte aligned.
			100	lbu	Load byte unsigned: same as lb but zero-extends instead of sign-extending.
			101	lhu	Load halfword unsigned: same as lh but zero-extends.
Stores	S	0100011	000	sb	Store byte: Mem[rs1+imm][7:0] = rs2[7:0]. Writes lowest 8 bits of rs2.
			001	sh	Store halfword: Mem[rs1+imm][15:0] = rs2[15:0]. Writes lowest 16 bits.
			010	sw	Store word: Mem[rs1+imm] = rs2. Writes full 32 bits.
Branches	B	1100011	000	beq	if (rs1 == rs2) PC += imm.
			001	bne	if (rs1 != rs2) PC += imm.
			100	blt	if (rs1 $<_s$ rs2) PC += imm.
			101	bge	if (rs1 $\geq_s$ rs2) PC += imm.
			110	bltu	if (rs1 $<_u$ rs2) PC += imm.
			111	bgeu	if (rs1 $\geq_u$ rs2) PC += imm.
Jump	J	1101111	—	jal	rd = PC+4; PC += sign_ext(imm). 21-bit offset, ±1 MiB range.
Jump reg.	I	1100111	000	jalr	rd = PC+4; PC = (rs1+sign_ext(imm)) & ~1. LSB cleared for alignment.
Upper imm.	U	0110111	—	lui	rd = imm[31:12] << 12. Upper 20 bits loaded, lower 12 zeroed.
PC + upper	U	0010111	—	auipc	rd = PC + (imm[31:12] << 12). PC-relative upper immediate.

Table 3: RV32I individual operations grouped by opcode class, showing how funct3 (column 2) selects the specific instruction within each group. For R-type shifts and add/sub, funct7 bit 5 provides further disambiguation. The notation $<_s$/$<_u$ denotes signed/unsigned comparison.

running a different program requires only writing new instructions to memory, not reconfiguring hardware. In contrast to dedicated hardware (e.g. a systolic array), a stored-program processor is general-purpose — the same circuit executes any program.

The **architectural state** of a RISC-V processor is the minimal information needed to fully determine what a program will do: the *register file* (32 × 32-bit registers), the *program counter* (PC), and the contents of *memory*. If the OS saves the architectural state at some point, it can interrupt the program, run something else, and later restore the state so the program continues as if nothing happened. The microarchitecture's job is to faithfully update the architectural state according to each instruction.

3.6 Compiling, Assembling, Loading

3.6.1 Toolchain overview

Translating a high-level program into a running binary involves four stages, shown in Figure 5. The **compiler** translates C into assembly language. The **assembler** converts assembly into machine code, producing an *object file* containing binary instructions and a symbol table. The **linker** combines one or more object files with library code and start-up code, resolves symbol addresses, and produces a single *executable*. Finally, the **loader** (typically the OS or a bootloader) copies the executable's text segment into memory and jumps to the entry point.

In practice, GCC performs all three translation steps in one command:

```
gcc -O1 -g prog.c -o prog       # compile + assemble + link

gcc -O1 -S prog.c -o prog.s     # compile only (view assembly)
gcc -c prog.s -o prog.o          # assemble only (object file)

objdump -S prog.o                # disassemble (machine + assembly + C)
```

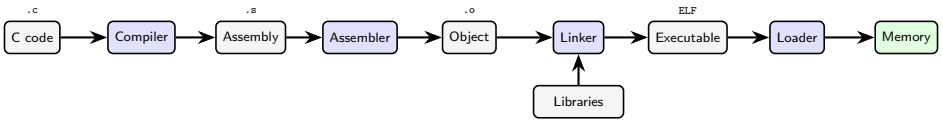


Figure 5: Program translation pipeline (after Harris Figure 6.30). Blue boxes are tools; grey boxes are intermediate file representations. The linker combines object files with libraries to produce the final executable, which the loader places into memory.

3.6.2 Memory map

With 32-bit addresses, the RISC-V address space spans 2^{32} bytes (4 GB). RISC-V does *not* define a fixed memory map — implementations are free to place segments wherever suits their hardware. Figure 6 shows the conventional layout used by Linux-class systems. The five segments serve distinct purposes:

- **Text**: the machine language program (instructions), plus literals and read-only data (`.rodata`).
- **Global data**: global variables allocated before the program runs. Accessed via the **global pointer** `gp` (register `x3`), which points to the middle of the segment so that a single 12-bit signed offset can reach the entire range.
- **Dynamic data**: the **stack** (grows downward from `sp`, used for function frames and local variables) and the **heap** (grows upward, used for `malloc`/`new` allocations). If they collide, the program's data is corrupted — the memory allocator prevents this by returning an out-of-memory error.
- **Exception handlers**: trap vectors and boot code at the lowest addresses.
- **OS and I/O**: operating system kernel and memory-mapped I/O peripherals at the highest addresses.

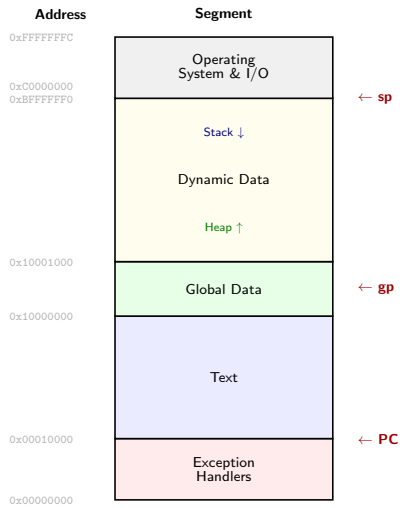


Figure 6: Example RISC-V memory map (after Harris Figure 6.31). Addresses increase upward. The PC initially points to the text segment; `gp` points to the middle of the global data segment; `sp` starts at the top of the stack. The heap grows upward and the stack grows downward within the dynamic data region.

3.7 Extensions and picorv32's instruction subset

RISC-V defines a base ISA plus optional extensions. The base ISA has 32-, 64-, and 128-bit base instruction sets: RV32I/E, RV64I, and RV128I. The picorv32 implements **RV32IMC**:[4]

Extension	Description	Status	picorv32
I	Base 32-bit integer (Table 2)	Frozen	Yes (core)
M	Integer multiply/divide	Frozen	Yes (via PCPI)
C	Compressed 16-bit instructions	Frozen	Yes (`COMPRESSED_ISA`)
F / D / Q	Single / double / quad floating-point	Frozen	No
A	Atomic instructions	Frozen	No
B	Bit manipulations	Open	No
V	Vector operations	Open	No

The **M extension** adds `mul`, `mulh`, `div`, `rem` and their unsigned variants. In picorv32 these are not part of the main ALU — they are implemented via the PCPI co-processor interface (Section 7.4), controlled by the `ENABLE_MUL`/`ENABLE_DIV` parameters. The **C extension** reduces common instructions to 16 bits by using 3-bit register codes (encoding only `x8`–`x15`, the 8 most-used registers) and shorter immediates. Typically 50–60% of instructions can be compressed, which is significant when instruction memory is limited to a few KB of EBR. The picorv32 decoder expands compressed instructions into their 32-bit equivalents before the main decode stage, so the rest of the datapath is unaware of RVC.

3.8 Architecture comparison

- **MIPS**: very similar (same ancestry). RISC-V improvements: no branch delay slot, PC-relative jumps, fixed `rs1`/`rs2`/`rd` bit positions across all formats.
- **ARM**: dominates mobile/embedded. Adds CISC-like features (conditional execution, multi-register push/pop, shifted source registers) to shrink code at the cost of decoder complexity. 16 GP registers vs RISC-V's 32.
- **x86**: CISC, variable-length instructions (1–15 bytes), only 8 GP registers, 2-operand format, condition flags, ALU can access memory directly. Modern x86 chips internally translate CISC instructions into RISC-like micro-operations.
- All four are load-store except x86. RISC-V and MIPS use compare-and-branch; ARM and x86 set flags first, then branch. RISC-V is the only open-source ISA with a modular extension system (M, F, C, V, ...).

[4] RISC-V defines three privilege levels (M/S/U-mode) and an exception mechanism using CSRs (`mtvec`, `mcause`, `mepc`, `mscratch`). When an exception occurs, the processor saves the PC in `mepc`, records the cause in `mcause`, and jumps to the handler at `mtvec`; `mret` returns. The picorv32 runs exclusively in M-mode and handles illegal instructions and misaligned accesses via the `trap` FSM state (bit 7), which raises an IRQ. See Harris §6.6.2 for full details.

4 Microarchitectures

In this chapter, we develop three microarchitectures for RISC-V: **single-cycle**, **multicycle**, and **pipelined**. They differ in how the state elements are connected and in the amount of nonarchitectural state needed. First we discuss the architecural state:

4.1 Architectural State and the Design Process

Recall from Section 3 that a computer architecture is defined by its instruction set and **architectural state**. For RISC-V, the architectural state consists of the *program counter* (PC) and the *32 general-purpose registers* (`x0`–`x31`). Any RISC-V microarchitecture must contain all of this state. If one were to save a copy of the architectural state and the contents of memory, then turn off a computer, then turn it back on and restore everything, the computer would resume the program it was running, unaware that it had ever been powered off.

Datapath and control. We divide every microarchitecture into two interacting parts:

- The **datapath** operates on words of data. It contains memories, registers, ALUs, and multiplexers.
- The **control unit** receives the current instruction from the datapath and tells the datapath how to execute it, producing multiplexer select, register enable, and memory write signals.

A good way to design a complex system is to start with hardware containing the **state elements** — the memories and architectural state (PC and registers). Then, add blocks of combinational logic between the state elements to compute the new state based on the current state. Since the instruction is read from one part of memory and load/store instructions access another, it is convenient to partition memory into instruction memory and data memory.

State elements. Figure 7 shows the four state elements:

- **Program counter (PC)**: a single 32-bit register. The `PCNext` input wire contains the address of next instruction. On each rising clock edge, it latches the value from this input into the register. Then after rising clock edge, its output wire contains the address of the current instruction.
- **Instruction memory**: read-only, single port. Takes a 32-bit address A and outputs the 32-bit instruction word at that address on RD.
- **Register file**: a 32-entry $\times$ 32-bit array with *two* read ports and *one* write port. The two read ports ($A1 \rightarrow RD1$, $A2 \rightarrow RD2$) supply both ALU operands simultaneously — this is why R-type instructions can read `rs1` and `rs2` in the same cycle. The write port ($A3$, $WD3$, $WE3$) writes data $WD3$ into register address $A3$ on the rising clock edge, but only when $WE3$ flag is asserted.
- **Data memory**: a single read/write port. If flag $WE = 0$, it reads: the word at address A appears on RD. If $WE = 1$, it writes: data WD is stored to address A on the rising clock edge.

All three memories are *read combinationally* — changing the address input causes new data to appear at the output after propagation delay, with no clock edge required. The clock controls *writing only*: the PC, register file, and data memory all update their stored state on the rising edge and hold it stable otherwise. This makes the processor a synchronous sequential circuit.

Colour convention. Throughout this document, diagrams use a consistent RGB colour scheme:[5]

Colour	Component	Examples
Blue	Memory / storage	Instruction memory, data memory, register file, SPRAM, cache
Green	Compute	ALU, DSP blocks, multipliers, PCPI modules
Red	Control / decode	Decoder, FSM, error/trap states
Gray	Intermediate registers / state	PC, nonarchitectural working registers

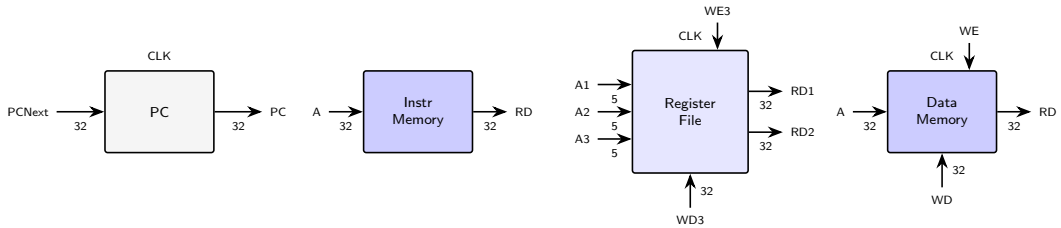


Figure 7: The four state elements of a RISC-V processor (after Harris Figure 7.1). The PC holds the current instruction address. The instruction memory is read-only (combinational). The register file has two read ports (A1/RD1, A2/RD2) and one clocked write port (A3/WD3/WE3). The data memory has one read/write port, clocked on writes.

4.2 Performance Analysis

The execution time of a program is given by the **performance equation**:

$$T_{\text{exec}} = N_{\text{instr}} \times \text{CPI} \times T_c$$

[5]Lighter shades distinguish sub-categories within the same family: e.g. darker blue for memory blocks, lighter blue for register files and buffers; darker green for the main ALU, lighter green for auxiliary compute units (e.g. PCPI multiplier).

- N_{instr} (**number of instructions**): depends on the ISA and the cleverness of the programmer/compiler. For a given program on a RISC-V processor, this is constant regardless of microarchitecture.
- **CPI** (**cycles per instruction**): the average number of clock cycles to execute one instruction. Its reciprocal is IPC (instructions per cycle).
- T_c (**clock period**): determined by the **critical path** through the logic. Different microarchitectures have different T_c because they put different amounts of logic between pipeline registers.

The challenge of the microarchitect is to minimise CPI $\times$ T_c while satisfying constraints on cost and power. Reducing one often increases the other: adding pipeline stages reduces T_c but may increase CPI due to hazards.

4.3 Single-cycle

A single-cycle processor executes every instruction in one clock cycle: CPI = 1. The datapath reads the instruction from memory, reads registers, performs an ALU operation, accesses data memory, and writes the result back — all in one combinational pass. The control unit is purely combinational (a truth table from opcode to control signals).

4.3.1 Datapath

The datapath is constructed incrementally from the state elements (Figure 7), adding hardware to handle each instruction class (Harris Figures 7.3–7.11):

1. **Fetch**: PC provides the address to instruction memory; the 32-bit output is the instruction word `Instr`.
2. **Register read**: `Instr[19:15]` (rs1) addresses register file port A1 into RD1, producing `SrcA` = RD1. For R-type/branches, `Instr[24:20]` (rs2) addresses port A2 into RD2, producing `SrcB`=RD2.
3. **Sign extension**: the Extend unit extracts the immediate field and sign-extends it to 32 bits. A 2-bit `ImmSrc` control signal selects which format (I/S/B/J) to extract.
4. **ALU**: computes `SrcA op SrcB`. The `ALUSrc` mux selects `SrcB` from either RD2 (R-type, branches) or `ImmExt` (I-type ALU, loads, stores). The 3-bit `ALUControl` selects the operation.[6]
5. **Data memory**: for `lw`, `ALUResult` is the address of where to load data from, and `ReadData` returns the loaded word. For `sw`, `MemWrite` flag is asserted, where RD2 feeds `WriteData` to the address at `ALUResult`.
6. **Writeback**: the `ResultSrc` mux selects `Result` from `ALUResult` (R-type, I-type ALU), `ReadData` (`lw`), or `PCPlus4` (`jal`). This feeds the data `Result` into the register file write port WD3 (gated by `RegWrite`), and writes into the address at A3.
7. **PC update**: one adder computes `PCPlus4` = PC+4. Another computes `PCTarget` = PC+`ImmExt` for branches/jumps. The `PCSrc` mux (driven by `Branch & Zero`, or `Jump`) selects `PCNext`.

Figure 8 shows the complete single-cycle processor. The key design strategy: compute *all* possible results in parallel (`ALUResult`, `ReadData`, `PCPlus4`, `PCTarget`), then use multiplexers (select bits driven by wires from control unit) to select the correct one. Hardware operates in parallel — unlike software, which computes sequentially. [7]

4.3.2 Control unit

The controller decomposes into a **Main Decoder** and an **ALU Decoder** (Figure 9). The Main Decoder maps `opcode[6:0]` to most control signals plus an internal 2-bit `ALUOp`. The ALU Decoder combines `ALUOp` with `funct3` (and sometimes $\texttt{op}_5$, $\texttt{funct7}_5$) to produce `ALUControl[2:0]`: [8]

4.3.3 Critical path

The clock period T_c is set by the slowest instruction (`lw`), whose critical path traverses: PC clk-to-Q $\rightarrow$ instruction memory $\rightarrow$ register file read $\rightarrow$ ALU $\rightarrow$ data memory $\rightarrow$ Result mux $\rightarrow$ register file setup:

$$T_{c,\text{single}} = t_{\text{pcq}} + 2\,t_{\text{mem}} + t_{\text{RFread}} + t_{\text{ALU}} + t_{\text{mux}} + t_{\text{RFsetup}}$$

Every instruction pays this worst-case penalty. The processor also needs separate instruction and data memories, since both are read in the same cycle.

4.4 Multicycle

A multicycle processor breaks each instruction into shorter steps (fetch, decode, execute, memory access, writeback), using one clock cycle per step. This has three advantages:

1. T_c is set by the slowest *step*, not the slowest instruction — typically a single memory access or ALU operation, roughly $\frac{1}{3}$ to $\frac{1}{5}$ of the single-cycle T_c. Hence T_c decreases.
2. Simple instructions finish in fewer cycles (branches: 3, R-type: 4, loads: 5), so average CPI is less than the worst case.

[6] The ALU only accepts 32-bit because that is the register width — both operands are always full 32-bit values (immediates are sign-extended before reaching the ALU). For data wider than 32 bits, software decomposes the operation: e.g. a 64-bit add uses two instructions (add lower halves, then add-with-carry upper halves). This overhead is why 32-bit ISAs eventually transition to 64-bit (ARMv7→AArch64, x86→x86-64).

[7] To support `jal`, the `ResultSrc` mux is expanded from 2:1 to 3:1 (adding `PCPlus4` as input 10), and a `Jump` signal is OR'd with (`Branch & Zero`) to drive `PCSrc` (Harris Figures 7.15–7.16).

[8] The ALU Decoder uses only 7 bits total: `ALUOp` (2 bits from the Main Decoder), all 3 bits of `funct3`, and two *individual* bits — $\texttt{op}_5$ (`instr[5]`, one bit from the opcode) and $\texttt{funct7}_5$ (`instr[30]`, one bit from funct7).

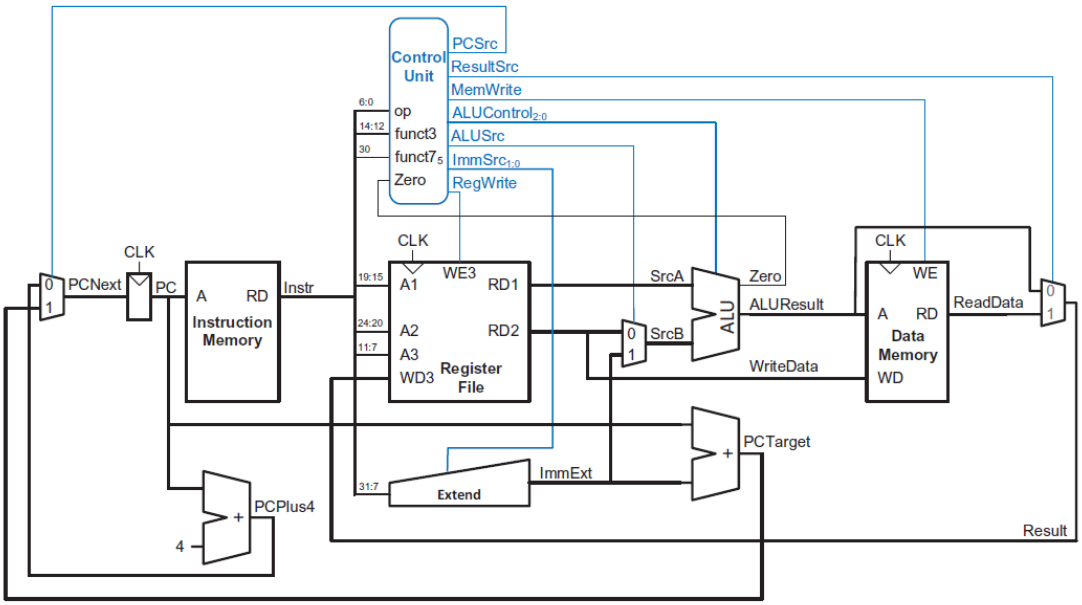


Figure 7.12 Complete single-cycle processor

Figure 8: Complete single-cycle RISC-V processor (Harris Figure 7.12). The datapath flows left to right: PC → instruction memory → register file → ALU → data memory. Blue signals are control outputs. The three key multiplexers — `ALUSrc` (selects RD2 or `ImmExt` as `SrcB`), `ResultSrc` (selects `ALUResult` or `ReadData`), and `PCSrc` (selects `PCPlus4` or `PCTarget`) — route data differently depending on the instruction type.

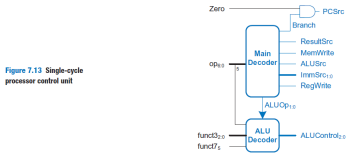


Figure 7.13 Single-cycle processor control unit

Instr	Opcode	RegW	ImmSrc	ALUSrc	MemW	ResSrc	Br	ALUOp	Jmp
`lw`	0000011	1	00	1	0	01	0	00	0
`sw`	0100011	0	01	1	1	xx	0	00	0
R-type	0110011	1	xx	0	0	00	0	10	0
`beq`	1100011	0	10	0	0	xx	1	01	0
I-type ALU	0010011	1	00	1	0	00	0	10	0
`jal`	1101111	1	11	x	0	10	0	xx	1

ALUOp	funct3	$\{op_5, f7_5\}$	ALUControl	Instruction
00	x	x	000 (add)	`lw`, `sw`
01	x	x	001 (sub)	`beq`
10	000	$\neq$11	000 (add)	`add`, `addi`
10	000	11	001 (sub)	`sub`
10	010	x	101 (slt)	`slt`, `slti`
10	110	x	011 (or)	`or`, `ori`
10	111	x	010 (and)	`and`, `andi`

Figure 9: Single-cycle control unit (Harris Figure 7.13). Left: the Main Decoder maps `opcode` to control signals + `ALUOp`; the ALU Decoder refines `ALUOp` using `funct3`/`funct7`$_5$. Right top: Main Decoder truth table (x = don't care). Right bottom: ALU Decoder truth table — `ALUOp` = 00 always adds, 01 always subtracts, 10 inspects `funct3`/`funct7`.

3. Hardware is **reused**: a single ALU handles address calculation, arithmetic, and PC increment on different cycles; a single memory serves both instructions and data.

The cost is a **finite state machine (FSM)** controller that sequences the control signals differently on each cycle, and **nonarchitectural registers** to hold intermediate results between steps.

4.4.1 Datapath

Figure 10 shows the complete multicycle processor. The key structural differences from the single-cycle design (Figure 8) are:

- **Unified memory**: a single memory serves both instructions and data. The `AdrSrc` multiplexer selects the address: the PC during Fetch state, or `ALUOut` (the computed data address) during MemRead/MemWrite.
- **Nonarchitectural registers** hold intermediate results between FSM steps:
  - `Instr`: Instruction register (IR): latched on Fetch state, holds the instruction word for later steps (cycles)
  - `OldPC`: saves the current PC address (before it is updated to PC+4), which is needed for branch/jump target computation.
  - `A` and `WriteData`: hold the register file outputs (rs1 and rs2) read during Decode.
  - `ALUOut`: stores the ALU result between steps (address, arithmetic result, or branch target).
  - `Data`: holds the word read from memory during MemRead.
- **Wider ALU source multiplexers**: because the ALU is reused for different purposes on different steps, the source muxes have more inputs. `ALUSrcA` selects among PC (for PC+4), `OldPC` (for branch targets), or `A` (for arithmetic/address calc). `ALUSrcB` selects among `WriteData` (R-type rs2), `ImmExt` (immediates), or the constant 4 (for PC+4).
- **Result multiplexer** (3 inputs): ALUOut (registered ALU results), Data (memory loads), or ALUResult (unregistered, used for PC+4 writeback during Fetch to save a cycle).

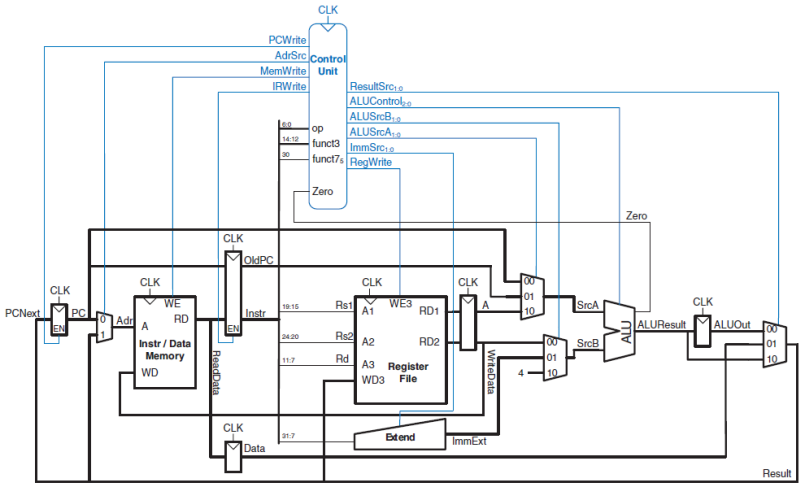


Figure 7.27 Complete multicycle processor

Figure 10: Complete multicycle RISC-V processor (Harris Figure 7.27). The datapath (black) uses a unified instruction/data memory and a single ALU, connected through nonarchitectural registers (Instr, OldPC, A, WriteData, ALUOut, Data). The control unit (blue) is an FSM that produces different control signals on each step. Compare with the single-cycle processor in Figure 8: the multicycle design eliminates two adders and one memory at the cost of six nonarchitectural registers and wider multiplexers.

4.4.2 FSM control

The controller is a Moore FSM with 11 states (Figure 11). All instructions share Fetch and Decode; after Decode, the opcode determines which path to take:

Instruction	State path	Cycles
`lw`	Fetch → Decode → MemAdr → MemRead → MemWB	5
`sw`	Fetch → Decode → MemAdr → MemWrite	4
R-type ALU	Fetch → Decode → ExecuteR → ALUWB	4
I-type ALU	Fetch → Decode → ExecuteI → ALUWB	4
`beq`	Fetch → Decode → BEQ	3
`jal`	Fetch → Decode → JAL → ALUWB	4

Table 4 details each state: the ALU mux selects (— when the ALU is idle), the micro-operation in register-transfer notation, and a description of what happens and why. States where the ALU is idle perform pure data routing — moving data between nonarchitectural registers, memory, and the register file via multiplexers and write-enable signals.

Data flow example: `lw`. Figure 12 traces data through the datapath during the first three steps of `lw`, with each step colour-coded:

#	State	SrcA	SrcB	Op	Micro-operation	Description
S0	Fetch	PC	4	add	`Instr ← Mem[PC]`	Read instruction from memory at the PC address and latch into IR.
					`PC ← PC+4`	ALU computes PC+4 and writes back to PC — reusing the ALU that is otherwise idle this cycle.
S1	Decode	OldPC	Imm	add	`A ← rs1`	Read source registers from register file into nonarchitectural registers.
					`WriteData ← rs2`	If dual-port, rs2 read simultaneously.
					`ALUOut ← OldPC+ImmExt`	ALU speculatively computes branch target. If not a branch, this result is unused.
S2	MemAdr	A	Imm	add	`ALUOut ← A + ImmExt`	Compute data memory address (base + offset). Shared by `lw` and `sw`.
S3	MemRead	—	—	—	`Data ← Mem[ALUOut]`	AdrSrc=1 routes ALUOut to memory address port. Data latched into Data register.
S4	MemWB	—	—	—	`rd ← Data`	ResultSrc=01 selects Data. RegWrite asserted; loaded value written to `rd`.
S5	MemWrite	—	—	—	`Mem[ALUOut] ← rs2`	AdrSrc=1 routes ALUOut to memory. MemWrite asserted; WriteData written to memory.
S6	ExecuteR	A	B	op	`ALUOut ← A` *op* `B`	R-type ALU operation. Specific op selected by funct3/funct7 via ALU Decoder.
S7	ALUWB	—	—	—	`rd ← ALUOut`	ResultSrc=00 selects ALUOut. RegWrite asserted. Shared by R-type, I-type, and `jal`.
S8	ExecuteI	A	Imm	op	`ALUOut ← A` *op* `ImmExt`	Same as ExecuteR but SrcB is sign-extended immediate. Handles `addi`, `andi`, `ori`, `slti`.
S9	JAL	OldPC	4	add	`PC ← ALUOut`	Write jump target (from Decode's speculative computation) to PC via Result mux.
					`ALUOut ← OldPC+4`	ALU computes return address OldPC+4, to be written to `rd` in next state (ALUWB).
S10	BEQ	A	B	sub	`if Zero: PC ← ALUOut`	ALU subtracts rs1−rs2. If zero (equal), Branch=1 asserts PCWrite; branch target (ALUOut from Decode) overwrites PC. Otherwise PC keeps PC+4 from Fetch.

Table 4: Complete multicycle FSM state table. SrcA/SrcB/Op show ALU mux selects (— = ALU idle). `\midrule` separates states; `\cline` separates micro-ops within a state.

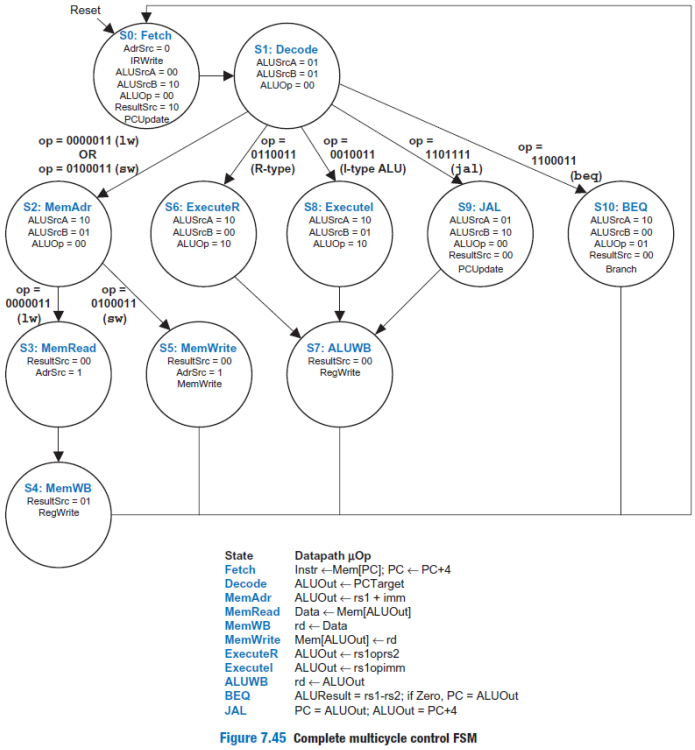


Figure 7.45 Complete multicycle control FSM

Figure 11: Complete multicycle control FSM with 11 states (Harris Figure 7.45). Fetch and Decode are shared by all instructions; subsequent states branch by opcode. The table summarises each state's micro-operation. Compare with picorv32's 8-state one-hot FSM (Figure 28): picorv32 merges Decode into `ld_rs1`, uses a single `exec` state for all ALU/branch/jump operations, and defers writeback to the next Fetch.

1. **Fetch** (grey): PC drives the memory address; the instruction word flows into IR. Simultaneously, the ALU adds PC + 4 and writes the result back to PC via the Result mux (ResultSrc = 10).
2. **Decode** (medium blue): `Instr[19:15]` (rs1) addresses the register file; the base address is latched into A. The Extend unit produces ImmExt from the I-type immediate field. The ALU speculatively computes OldPC + ImmExt for a potential branch target.
3. **MemAdr** (dark blue): the ALU adds A + ImmExt (ALUSrcA = 10, ALUSrcB = 01, ALUOp = 00) to produce the memory address, stored in ALUOut.
4. **MemRead**: the AdrSrc mux switches to 1, routing ALUOut (the computed address) to the memory address input. The memory reads from this address and the result is latched into the Data nonarchitectural register at the end of the cycle.
5. **MemWB**: the Result mux selects Data (ResultSrc = 01), placing the loaded word on the register file's WD3 input. RegWrite is asserted, so at the next rising clock edge the value is written into the destination register rd (specified by `Instr[11:7]`).

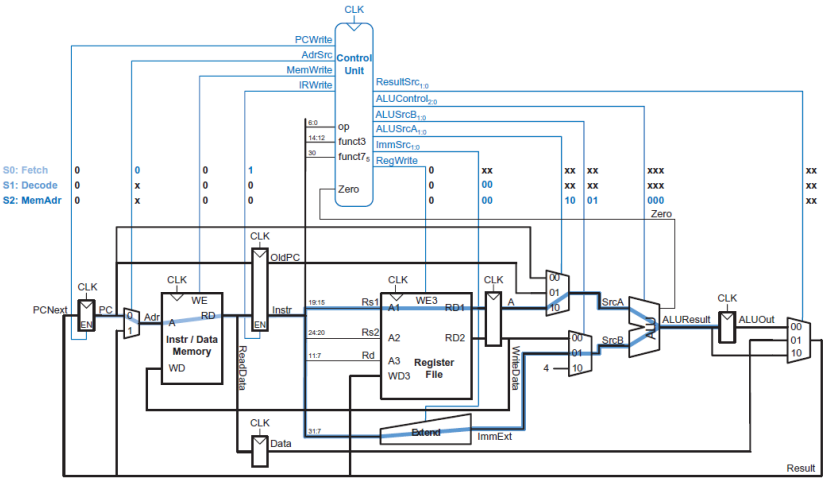


Figure 7.32 Data flow during Fetch, Decode, and MemAdr states

Figure 12: Data flow during Fetch, Decode, and MemAdr for `lw` (Harris Figure 7.32). Grey: Fetch (instruction read + PC+4). Medium blue: Decode (register read + speculative branch target). Dark blue: MemAdr (base + offset address computation). The control signal values for each state are shown in the table at top.

4.4.3 Critical path

The cycle time is set by the slower of two paths (Harris Figure 7.46): (1) the PC+4 path through the SrcA mux, ALU, and Result mux back to the PC register; or (2) the memory read path through the Result and Adr muxes to memory into the Data register. Both paths include a decoder delay after the FSM state updates:

$$T_{c,\text{multi}} = t_{\text{pcq}} + t_{\text{dec}} + 2\,t_{\text{mux}} + \max[t_{\text{ALU}},\ t_{\text{mem}}] + t_{\text{setup}}$$

With the 7 nm delays from Harris Table 7.7, $T_{c,\text{multi}} = 40 + 25 + 2(30) + 200 + 50 = 375$ ps — half the single-cycle T_c of 750 ps. But the average CPI of 4.14 (SPECINT2000 mix) more than cancels this gain: total execution time T_{exec} is 155 s versus 75 s for single-cycle. The fundamental problem is that splitting `lw` into 5 steps does not reduce T_c fivefold: the steps are unequal in length, and the 90 ps sequencing overhead ($t_{\text{pcq}} + t_{\text{setup}}$) is paid on *every* step.

The multicycle processor's advantage is hardware cost: one memory instead of two, one ALU instead of three adders, no pipeline registers, no hazard unit. This matters on the iCE40UP5K with only 5280 LUTs and 15 KB of EBR — which is exactly why picorv32 uses a multicycle architecture.

4.5 Pipelined

A pipelined processor subdivides the single-cycle datapath into five stages separated by pipeline registers. Five instructions execute simultaneously, one in each stage. Because each stage has only one-fifth of the logic, the clock frequency is approximately five times faster. Ideally the latency of each instruction is unchanged, but the throughput is five times better. Pipelining introduces some overhead (sequencing, hazards), so the speedup is less than five in practice — but the advantage is so large that all modern high-performance processors are pipelined.

Pipeline stages and operation. The five stages each involve exactly one of the slow hardware blocks (memory read, register file, or ALU):

1. **Fetch (F)**: read the instruction from instruction memory at the address in PC.
2. **Decode (D)**: read source operands from the register file and decode the instruction to produce control signals.
3. **Execute (E)**: the ALU performs the computation (arithmetic, address calculation, or branch comparison).
4. **Memory (M)**: read or write data memory for loads and stores.
5. **Writeback (W)**: write the result back to the register file.

Figure 13 shows six instructions flowing through the pipeline. Reading across a row shows when a particular instruction occupies each stage; reading down a column shows what all five stages are doing on a given cycle. For example, in cycle 6, the register file is writing a sum to `s3`, the data memory is idle, the ALU is computing `s11 & t0`, `t4` is being read from the register file, and the `or` instruction is being fetched. The instruction memory and ALU are active every cycle; the data memory is used only by loads and stores.

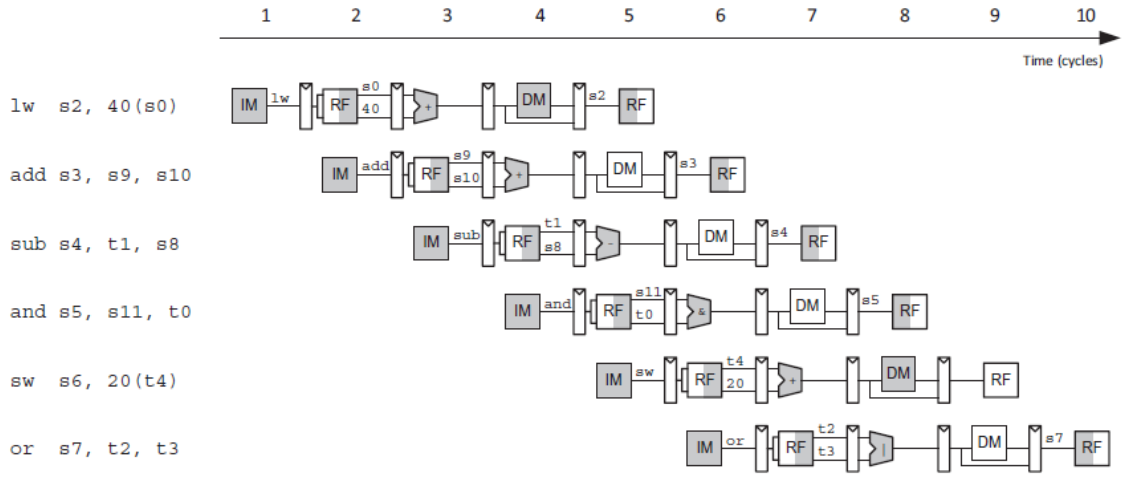


Figure 7.48 **Abstract view of pipeline in operation**

Figure 13: Abstract view of the pipeline in operation (Harris Figure 7.48). Each stage is represented by its major component: IM (instruction memory), RF (register file read), ALU, DM (data memory), RF (register file writeback). The register file is written in the first half of the cycle and read in the second half, so data can be written by one instruction and read by the next within a single cycle.

4.5.1 Datapath

The pipelined datapath is formed by inserting four pipeline registers into the single-cycle datapath (Figure 8), separating it into five stages. Figure 14 shows the result. Signals carry a suffix (F, D, E, M, or W) indicating the stage in which they reside.

Two subtleties are critical:

- The register file writes on the *falling* edge of CLK (first half of the cycle) and is read on the rising edge (second half). This allows a result to be written by one instruction and read by the next within the same cycle without introducing a hazard.
- The destination register `Rd` must be *pipelined through* the Execute, Memory, and Writeback stages so it stays synchronised with the instruction's data. If `Rd` were taken from the Decode stage (as in the single-cycle processor), it would correspond to the wrong instruction.

4.5.2 Control

The pipelined processor uses the same control unit as the single-cycle processor — the same Main Decoder and ALU Decoder from Tables 9 and 9. The control signals are produced in the Decode stage and pipelined along with the data through subsequent stages. For example, `RegWriteD` becomes `RegWriteE` in Execute, `RegWriteM` in Memory, and `RegWriteW` in Writeback, where it finally enables the register file write.

4.5.3 Hazards

When multiple instructions execute simultaneously, dependencies arise that the hardware must resolve. These are classified as **data hazards** (an instruction needs a register value that a prior instruction has not yet written back) and **control hazards** (the branch decision has not been made by the time the next instruction must be fetched). A dedicated **Hazard Unit** detects these situations and resolves them using three mechanisms: forwarding, stalling, and flushing.

Solving data hazards with forwarding. A *read after write* (RAW) hazard occurs when an instruction in the Execute stage reads a register that a preceding instruction has not yet written back, hence reading the stale value. **Forwarding** (also called bypassing) solves this by routing the result directly from a later pipeline stage (Memory or Writeback) to the ALU inputs in Execute, bypassing the register file entirely. The hardware cost is two 3:1 forwarding multiplexers in front of the ALU, selecting each operand from: (00) the register file output, (01) `ResultW` from the Writeback stage, or (10) `ALUResultM` from the Memory stage. The Hazard Unit compares the source registers of the instruction in Execute (`Rs1E`, `Rs2E`) against the destination registers in Memory and Writeback (`RdM`, `RdW`) to determine which input to select:

```
if   ((Rs1E == RdM) & RegWriteM & (Rs1E != 0))  ForwardAE = 10  // Memory
elif ((Rs1E == RdW) & RegWriteW & (Rs1E != 0))  ForwardAE = 01  // Writeback
else                                            ForwardAE = 00  // Register file
```

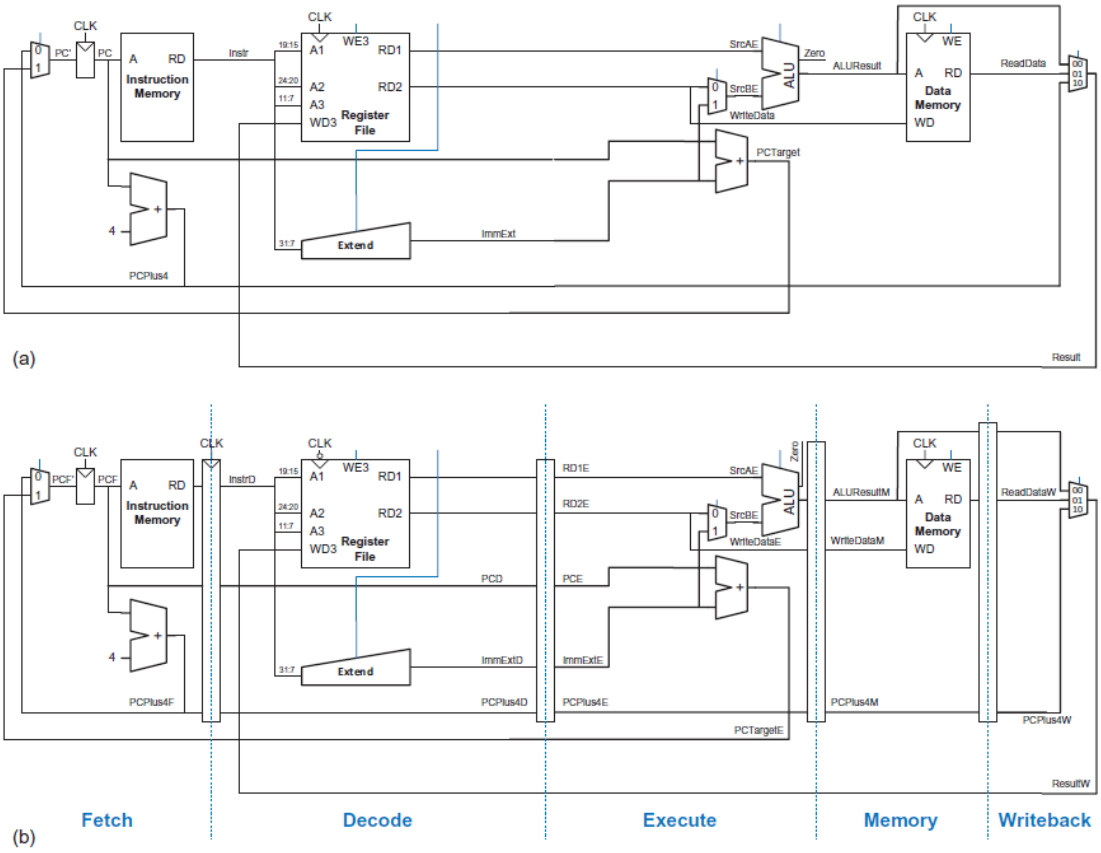


Figure 7.49 Datapaths: (a) single-cycle and (b) pipelined

Figure 14: Pipelined datapath (Harris Figure 7.49b). Four pipeline registers (vertical bars) separate the five stages. The register file is drawn in the Decode stage, but its write address (`RdW`) and write data (`ResultW`) come from the Writeback stage — this feedback is the source of pipeline hazards.

The logic for `ForwardBE` (the second ALU operand) is identical but checks `Rs2E`. The Memory stage has priority over Writeback because it contains the more recently executed instruction. Register `x0` is never forwarded because it is hardwired to zero.

Solving data hazards with stalls. Forwarding works when the result is computed in the Execute stage, because it can be forwarded from Memory to the next instruction's Execute. But `lw` does not produce its result until the end of the Memory stage (when data memory is read), so the result cannot be forwarded to an instruction that needs it in the same cycle's Execute stage. This is a **load-use hazard**: `lw` has a two-cycle latency. The solution is to *stall* the pipeline for one cycle, inserting a *bubble* (a do-nothing cycle that propagates through the pipeline like a `nop`). The Fetch and Decode pipeline registers are held (`EN`=0, freezing their contents), and the Execute pipeline register is cleared (`CLR`=1, zeroing all control signals). After the stall, the `lw` result has reached the Writeback stage and can be forwarded normally. The stall condition is: $\texttt{lwStall} = \texttt{ResultSrcE}_0 \ \texttt{\&} \ ((\texttt{Rs1D} = \texttt{RdE}) \,|\, (\texttt{Rs2D} = \texttt{RdE}))$ where $\texttt{ResultSrcE}_0 = 1$ indicates a load word is in the Execute stage.

Solving control hazards with flushing. The `beq` instruction presents a control hazard: the branch decision (`PCSrcE`) is not computed until the Execute stage, by which time two subsequent instructions have already been fetched into the Decode and Execute stages. The simplest strategy is **predict not-taken**: the processor continues fetching sequentially and, if the branch turns out to be taken, *flushes* (discards) the two incorrectly fetched instructions by clearing their pipeline registers (`CLR`=1). This incurs a 2-cycle misprediction penalty on every taken branch.

Hazard summary. Figure 15 shows the complete pipelined processor with the Hazard Unit. The full hazard logic is:

Signal	Logic
`ForwardAE`	10 if `(Rs1E==RdM) & RegWriteM & (Rs1E!=0)`; 01 if `(Rs1E==RdW) & RegWriteW & (Rs1E!=0)`; else 00
`lwStall`	$\texttt{ResultSrcE}_0$ `& ((Rs1D==RdE) | (Rs2D==RdE))`
`StallF, StallD`	`lwStall`
`FlushD`	`PCSrcE`
`FlushE`	`lwStall | PCSrcE`

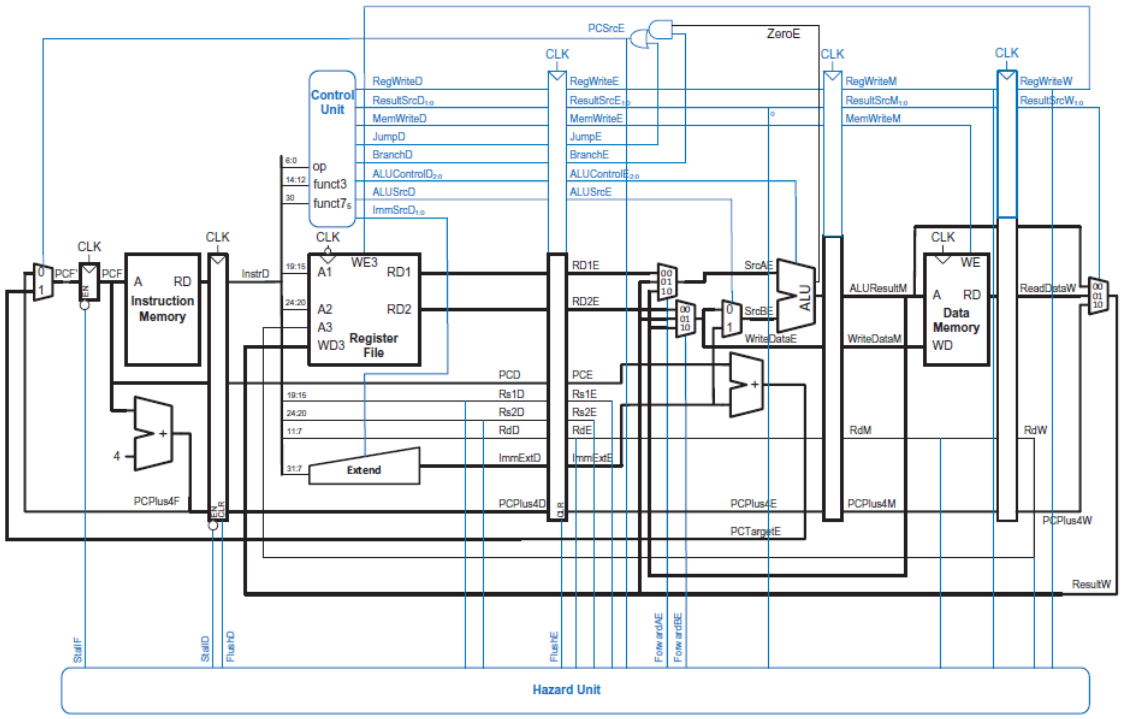


Figure 7.61 Pipelined processor with full hazard handling

Figure 15: Complete pipelined RISC-V processor with full hazard handling (Harris Figure 7.61). The Hazard Unit (bottom) receives source registers from Execute (`Rs1E`, `Rs2E`) and destination registers from Memory and Writeback (`RdM`, `RdW`) to compute forwarding signals. It also receives $\texttt{ResultSrcE}_0$ and `PCSrcE` to compute stall and flush signals. Compare with the single-cycle processor (Figure 8): the pipeline adds four pipeline registers (vertical bars between stages), two forwarding multiplexers (3:1 muxes before the ALU), and enable/clear inputs on the pipeline registers for stall/flush control.

4.5.4 Critical path

The cycle time is set by the slowest of the five pipeline stages. Each stage's critical path must also account for pipeline register sequencing overhead (t_{pcq} and t_{setup}). The Decode and Writeback stages use the register file in half-cycles (write on falling edge, read on rising edge), so twice their critical path must fit in one cycle:

$$T_{c,\text{pipe}} = \max \begin{cases} t_{\text{pcq}} + t_{\text{mem}} + t_{\text{setup}} & \text{Fetch} \\ 2(t_{\text{RFread}} + t_{\text{setup}}) & \text{Decode} \\ t_{\text{pcq}} + 4\,t_{\text{mux}} + t_{\text{ALU}} + t_{\text{AND-OR}} + t_{\text{setup}} & \text{Execute} \\ t_{\text{pcq}} + t_{\text{mem}} + t_{\text{setup}} & \text{Memory} \\ 2(t_{\text{pcq}} + t_{\text{mux}} + t_{\text{RFsetup}}) & \text{Writeback} \end{cases}$$

The Execute stage is the bottleneck: its critical path includes the forwarding mux chain (Result mux → ForwardBE mux → SrcB mux), the ALU, and the AND-OR logic that computes `PCSrcE` from `BranchE & ZeroE | JumpE`.

The CPI is ideally 1 but increases due to hazard penalties: load-use stalls add 1 cycle per dependent load, and branch mispredictions flush 2 cycles per taken branch. With a realistic instruction mix (SPECINT2000), average CPI ≈ 1.25. With 7 nm delays, $T_{c,\text{pipe}} = 350$ ps (limited by the Execute stage), giving $1.7\times$ speedup over single-cycle despite being far from the ideal $5\times$ (Table 5). The gap comes from unbalanced stages and sequencing overhead paid on every stage. The hardware cost is pipeline registers, forwarding multiplexers, and the Hazard Unit.

4.6 Summary

The three architectures in Figure 16 differ in how they connect the same building blocks:

- **(a) Single-cycle**: data flows left-to-right through the entire datapath in one clock cycle. The critical path spans PC → instruction memory → register file → ALU → data memory → writeback (red dashed line), so T_c is long. CPI = 1, but the clock is slow. Requires two separate memories (instruction and data, since both are read in the same cycle) and three adders (ALU + two for PC logic). The controller is purely combinational — a truth table from opcode to control signals.
- **(b) Multicycle**: the same hardware is reused across multiple shorter cycles. A single unified memory serves both instructions and data (accessed on different cycles), and a single ALU handles arithmetic, address calculation, and PC+4 (on different cycles). The dashed feedback arrows show data cycling back: ALU output feeds back to the memory address input and to the register file. Nonarchitectural registers (IR, A, B, Out) hold intermediate results between steps. The controller is an FSM that produces different control signals on each step. T_c is fast (set by a single memory access or ALU operation), but CPI = 3–5.

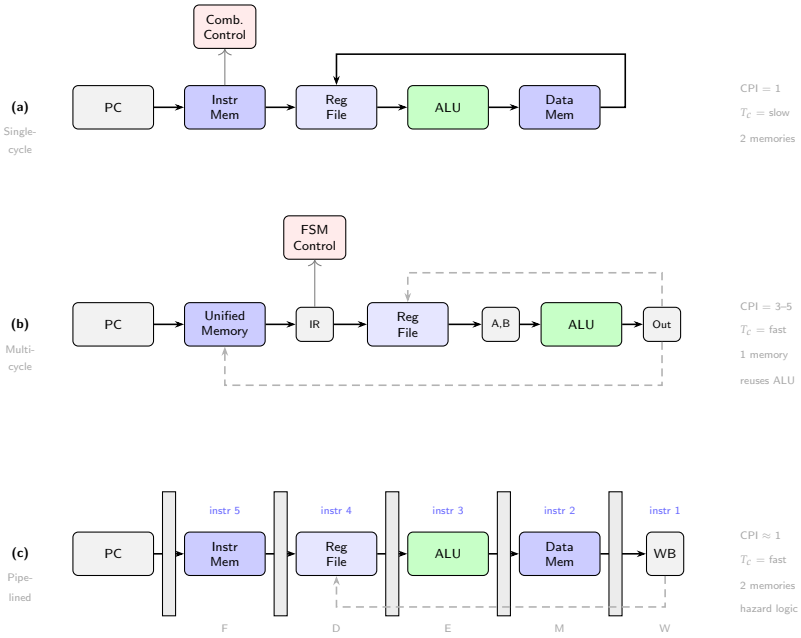


Figure 16: Three microarchitecture styles, compared below. For full datapaths with all control signals, see Harris Figures 7.12, 7.27, and 7.61.

- **(c) Pipelined**: the single-cycle datapath is chopped into five stages by inserting pipeline registers (grey bars). Five instructions execute simultaneously, one in each stage (labelled F, D, E, M, W). Like the single-cycle design, it needs two memories and has CPI ≈ 1, but T_c is set by the slowest *stage* rather than the entire datapath. The cost is hazard logic: forwarding multiplexers, stall/flush signals, and a Hazard Unit to handle data and control dependencies between instructions in different stages.

Harris Example 7.10 compares the three architectures executing 10^{11} instructions:

	CPI	T_c (ps)	T_{exec} (s)	Memories
Single-cycle	1	750	75	2 (separate I/D)
Multicycle	4.14	375	155	1 (shared)
Pipelined	1.25	350	44	2 (I/D caches)

Table 5: Performance comparison of three microarchitectures (Harris Example 7.10, 7 nm CMOS delays).

The multicycle processor is actually the *slowest* here because its high CPI (4.14) overwhelms its shorter T_c. The pipelined processor wins with $1.7\times$ speedup over single-cycle. But the multicycle design uses the *least hardware* (one memory, one ALU, no pipeline registers, no hazard unit), which matters when the FPGA has only 5280 LUTs.

4.7 Advanced microarchitecture techniques

Beyond the three basic microarchitectures, high-performance processors use additional techniques to increase throughput. Harris §7.7 surveys these; the most relevant to ML accelerator design are summarised here.

Superscalar processors (spatial parallelism). A superscalar processor contains multiple copies of the datapath hardware to execute multiple instructions per cycle. A two-way superscalar fetches two instructions, reads four source operands from a six-ported register file, executes on two ALUs, and writes two results — achieving IPC > 1 on code without dependencies. Commercial processors are three- to six-way superscalar, but real programs have enough dependencies that the execution units are rarely fully utilised. GPUs take this idea to the extreme: thousands of simple cores executing the same instruction on different data (SIMT).

Multiprocessing and multithreading. **Multiprocessing** places multiple independent processor cores on a single chip, each with its own datapath and control unit. Each core can run a separate program simultaneously. A quad-core CPU has four complete processors sharing main memory; coordination between cores uses shared-memory protocols (cache coherence) or message passing.

Hardware multithreading is a different idea: a *single* core replicates only the architectural state (PC + register file) so multiple threads are resident simultaneously, but they *share* the datapath hardware (ALU, memory interface). When one thread stalls on a slow memory access, the core immediately switches to another thread whose registers are already loaded — no context-switch penalty. The datapath stays busy even though individual threads are waiting. *fine-grained* multithreading switches threads every cycle (round-robin), smoothing out short stalls; *coarse-grained* switches only on expensive stalls like cache misses, avoiding the per-cycle switching overhead. **Simultaneous multithreading** (SMT, marketed as Intel Hyper-Threading) goes further: multiple threads issue instructions to the *same* execution units in the *same* cycle, filling bubbles left by dependencies in each individual thread.

The key distinction: multiprocessing duplicates entire cores (more hardware, true parallel execution); multithreading duplicates only state (less hardware, hides latency by interleaving). GPUs combine both: thousands of simple cores,

each running many hardware threads (warps), to hide the long latency of DRAM accesses.

Hardware accelerators. Rather than adding more identical cores, a system can incorporate specialised hardware (accelerators) optimised for specific tasks. Accelerators can be 10–100× more efficient in performance, cost, and power than general-purpose cores for the same computation. Examples include GPU shader cores for graphics, DSPs for signal processing, and TPU-style systolic arrays for matrix multiply. The CPU handles control flow and data movement; the accelerator handles the data-parallel inner loop.

4.8 Where picorv32 fits

Regardless of microarchitecture, a processor presents the same external interface: address and data busses to instruction and data memories, plus clock and reset. Figure 17 shows this for the single-cycle case — the Controller and Datapath are encapsulated in a module that communicates with external memories through well-defined ports (PC, Instr, DataAdr, WriteData, ReadData). A multicycle or pipelined processor has a different internal organisation but exposes the same kind of interface. picorv32 follows this pattern, replacing the direct address/data busses with a valid/ready handshake (`mem_valid`/`mem_ready`).

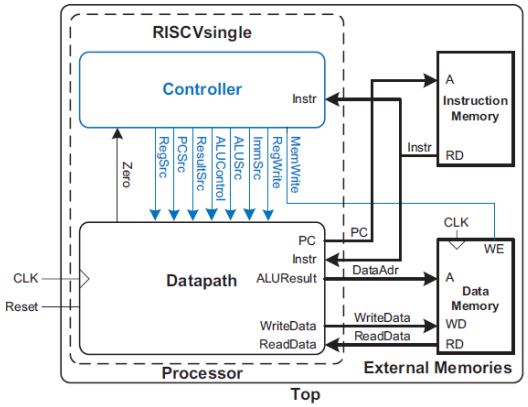


Figure 7.63 Single-cycle processor interfaced to external memories

Figure 17: Single-cycle processor interfaced to external memories (Harris Figure 7.63). The processor module contains a Controller and Datapath; it communicates with separate Instruction and Data memories through address, data, and control busses. Any microarchitecture (single-cycle, multicycle, pipelined) presents a similar external interface — only the internal organisation differs.

The picorv32 is a multicycle design — it uses a single memory interface, an FSM controller, and nonarchitectural registers — but it departs from the textbook in several ways:

- *One-hot state encoding*: 8 states in an 8-bit register (`cpu_state[7:0]`), each state a single bit. This synthesises to fast parallel-case logic on FPGAs, unlike the textbook's binary-encoded FSM.
- *No dedicated decode state*: the textbook multicycle FSM has a Decode state that reads registers and computes the branch target address. picorv32 merges this into `cpu_state_ld_rs1`, saving one cycle on most instructions.
- *Configurable datapath*: parameters like `BARREL_SHIFTER`, `TWO_CYCLE_ALU`, `TWO_STAGE_SHIFT` let the user trade LUTs for speed. Without a barrel shifter, shifts take 1–32 extra cycles; with one, they complete in the ALU combinationally.
- *Look-ahead memory interface*: the `mem_la_*` signals allow memory to begin responding before the bus handshake is formally asserted, partially overlapping fetch with decode.
- *PCPI co-processor port*: unrecognised instructions are forwarded to external hardware, enabling multiply/divide without enlarging the core.

5 Memory Systems

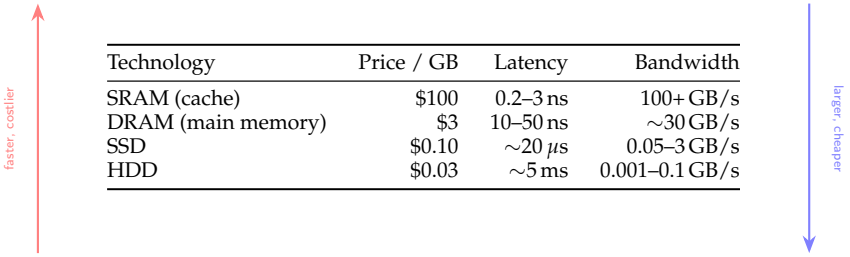
Chapter 7 assumed an ideal memory that could be accessed in a single clock cycle. In reality, DRAM is 10–100× slower than the processor. Memory systems bridge this gap using a hierarchy of progressively larger, slower, cheaper storage, exploiting the locality of real programs to make the common case fast.

5.1 The Memory Hierarchy

Memory speed is much slower than processor speed. DRAM throughput is reasonable (∼30 GB/s), but latency is the bottleneck. The solution exploits two empirical principles:

- **Temporal locality**: if a piece of data was accessed recently, it is likely to be accessed again soon. (A variable used in a loop is reused every iteration.)
- **Spatial locality**: if a piece of data was accessed, nearby data is likely to be accessed soon. (Consecutive array elements are stored at consecutive addresses.)

By keeping recently and nearby used data in a small, fast memory (a **cache**, built from on-chip SRAM), and storing the rest in large, slow main memory (DRAM) or an even larger hard drive, the system approximates a memory that is simultaneously fast, large, and cheap. Figure 6 shows the hierarchy with representative 2021 characteristics.



Technology	Price / GB	Latency	Bandwidth
SRAM (cache)	\$100	0.2–3 ns	100+ GB/s
DRAM (main memory)	\$3	10–50 ns	∼30 GB/s
SSD	\$0.10	∼20 μs	0.05–3 GB/s
HDD	\$0.03	∼5 ms	0.001–0.1 GB/s

Table 6: Memory hierarchy (after Harris Figure 8.3, Table 8.1). Each level is roughly 10× slower and 10–30× cheaper per GB than the one above. The processor seeks data in the cache first; on a miss, it falls through to main memory, then to disk.

If the processor requests data that is available in the cache, it is returned quickly — a **cache hit**. Otherwise, the data must be fetched from main memory — a **cache miss**. If the cache hits most of the time, the average access time approaches the cache speed rather than the DRAM speed. The hard drive provides **virtual memory**: an illusion of more capacity than physically exists in DRAM. Main memory acts as a cache for the hard drive, just as the cache acts as a cache for main memory.

5.2 Memory System Performance Analysis

The key metrics are **miss rate** (or equivalently hit rate) and **average memory access time** (AMAT):

$$\text{Miss Rate} = \frac{\text{Number of misses}}{\text{Total memory accesses}} = 1 - \text{Hit Rate}$$

$$\text{AMAT} = t_{\text{cache}} + \text{MR}_{\text{cache}}(t_{\text{MM}} + \text{MR}_{\text{MM}} \cdot t_{\text{VM}})$$

where t_{cache}, t_{MM}, t_{VM} are the access times of the cache, main memory, and virtual memory, and MR_{cache}, MR_{MM} are their miss rates. For a two-level system (cache + main memory only), this simplifies to $\text{AMAT} = t_{\text{cache}} + \text{MR}_{\text{cache}} \cdot t_{\text{MM}}$.

Amdahl's Law. Improving one subsystem yields diminishing returns if that subsystem is not the dominant bottleneck. If 50% of instructions are loads/stores, making memory 10× faster improves overall performance by only $1/(0.5 + 0.5/10) = 1.82\times$. The effort spent increasing a subsystem's performance is worthwhile only if that subsystem affects a large fraction of the overall execution time.

5.3 Caches

A cache holds a subset of main memory's data. It is specified by five parameters:

- **Capacity** C: total number of data words the cache can hold.
- **Block size** b: number of words per cache block (also called a cache line). A block is the unit of transfer between cache and main memory.
- **Number of blocks** $B = C/b$.
- **Associativity** N: number of blocks in each set (i.e., how many places a given address can map to).
- **Number of sets** $S = B/N$.

5.3.1 What Data is Held in the Cache?

The cache exploits both forms of locality:

- **Temporal locality**: when the processor loads or stores data not in the cache, that data is copied from main memory into the cache. Subsequent accesses to the same address hit in the cache.
- **Spatial locality**: when a cache miss occurs, the cache fetches not just the requested word but an entire **block** of b adjacent words. Subsequent accesses to nearby addresses are likely to hit because the neighbouring words are already in the cache.

If a program accessed data in a purely random order, a cache would provide no benefit. But real programs have strong locality — variables are reused in loops (temporal), and array elements are accessed sequentially (spatial).

5.3.2 How is Data Found?

The cache is organised as a two-dimensional array: the rows are **sets** and the columns are **ways**. Each memory address maps to exactly one set (determined by some address bits), but within that set, the data can be stored in any of the N ways.

Direct mapped cache. A direct mapped cache has $N = 1$ (one block per set, so $S = B$). Each address maps to a *unique* location in the cache. Figure 18 shows the mapping for an 8-word cache (8 sets) with $b = 1$: the bottom two bits of the address are always 00 (word-aligned[9]); the next $\log_2 8 = 3$ bits select the set; higher addresses wrap around, so addresses 0x04, 0x24, 0x44, … all map to set 1.

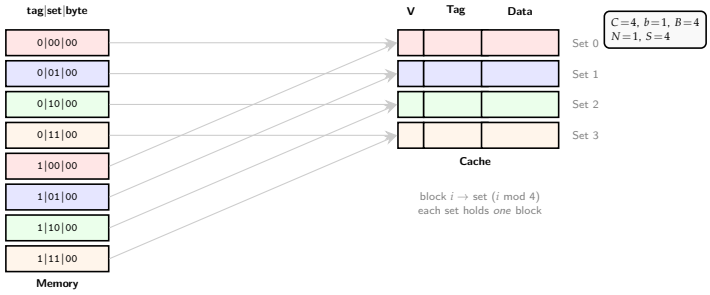
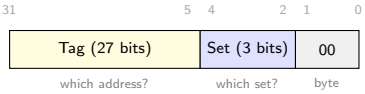


Figure 18: Direct mapped cache ($N = 1$, $S = 4$). Binary addresses show `tag|set|byte` fields; blocks with the same set bits (e.g. 0 and 4, both 00) conflict.

Because many addresses map to the same set, the cache must record *which* address is actually stored there. The main memory address is broken down into three fields:



The tag is only 27 bits, not the full 32-bit address, because the other 5 bits are already known: the set bits tell the hardware *which row* it is reading, and the byte offset is implied by word alignment. Together, {`tag, set, 00`} reconstructs the full 32-bit address. Each set in the cache stores:

- **Byte offset** [1:0]: always 00 for word-aligned accesses (selects the byte within the word). Not stored in the cache — implicit.
- **Set bits** $[\log_2 S + 1 : 2]$: select which row in the cache SRAM to read. Not stored — implicit from the row position.
- **Tag** $[31 : \log_2 S + 2]$: the remaining upper bits (27 in this example). Stored alongside the data so the cache can check whether the requested address matches what is actually held in this set.
- **Valid bit** V (1 bit): is this entry holding real data? $V = 0$ after power-on means the entry is empty.
- **Data** (b words): the actual cached data.

A **cache hit** occurs when $V = 1$ *and* the stored tag matches the tag field of the requested address. Figure 19 shows the hardware: the set bits index an 8-entry SRAM of $(1 + 27 + 32)$-bit rows; the stored tag is compared with the address tag; if they match and $V = 1$, the data is returned.

Example: temporal locality (Harris 8.6). Consider a loop that executes 5 iterations, each accessing addresses 0x4, 0xC, and 0x8 (mapping to sets 1, 3, 2). On the first iteration, the cache is empty: 3 misses. On iterations 2–5, all accesses hit. Miss rate = $3/15 = 20\%$.

Example: conflict misses (Harris 8.7). Now consider a loop accessing addresses 0x4 and 0x24 — both map to set 1. On every iteration, each access evicts the other. The miss rate is 100%, even though the cache has 8 sets and only 2 are needed. This pathology is a **conflict miss**: two addresses compete for the same set.

[9] **Word-aligned** means the address is a multiple of 4 (the word size in bytes), so bits [1:0] are always 00. This ensures a 32-bit word is read in a single memory access. Since memory is byte-addressed, a 32-bit word spans 4 consecutive byte addresses.

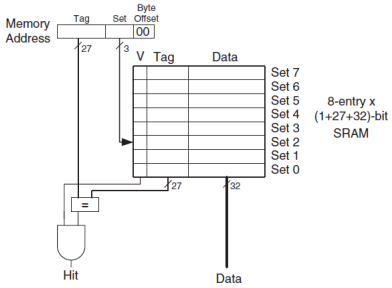


Figure 8.7 Direct mapped cache with 8 sets

Figure 19: Direct mapped cache hardware (Harris Figure 8.7). The set bits index an 8-entry SRAM; the stored tag is compared with the address tag; if they match and $V = 1$, the cache hits and returns the data.

Set associative cache. An N-way set associative cache provides N blocks per set ($S = B/N$), reducing conflicts. The address still maps to a unique set, but within that set, the data can occupy any of the N ways. Figure 20 shows the mapping for a 2-way cache: compared to direct-mapped, the number of sets halves (from 4 to 2) but each set can hold two blocks, so the total capacity is the same. The hardware reads all N tags in the selected set simultaneously and checks for a match. Figure 21 shows the hardware for $C = 8$ words.

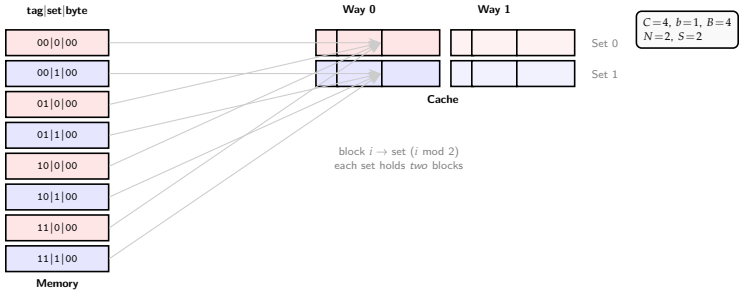


Figure 20: 2-way set associative cache ($N=2$, $S=2$). The set field is now 1 bit; the tag grows to 2 bits. Blocks 0 and 4 (both set bit 0) coexist in different ways instead of conflicting.

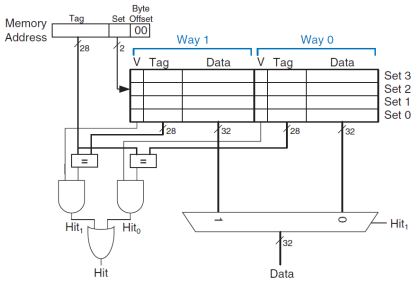


Figure 8.9 Two-way set associative cache

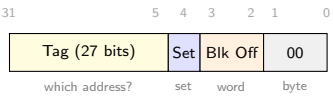
Figure 21: Two-way set associative cache hardware (Harris Figure 8.9). The cache has $S = 4$ sets (2 set bits instead of 3). Each set contains two ways, each with its own tag and valid bit. Both ways are read and compared in parallel; a multiplexer selects the data from the hitting way.

With 2 ways, the conflict from the previous example disappears: addresses 0x4 and 0x24 both map to set 1, but they occupy different ways. The miss rate drops from 100% to $2/10 = 20\%$ (2 compulsory misses on the first iteration, then all hits). Set associative caches generally have lower miss rates than direct mapped caches of the same capacity. The cost is N tag comparators and an output multiplexer, making them slower and slightly larger. Most commercial systems use 4-way or 8-way set associative caches.

Fully associative cache. A fully associative cache has $S = 1$ set and B ways — data can go in any block (Figure 22). This minimises conflict misses but requires B comparators, making it practical only for very small caches (e.g. TLBs with 16–512 entries).

Block size. The examples above used $b = 1$ (one word per block), exploiting only temporal locality. To exploit spatial locality, caches use larger blocks ($b > 1$): when a miss occurs, the entire block of b adjacent words is fetched from main memory. Subsequent accesses to nearby addresses hit because those words are already in the cache. With $b > 1$, the address gains a **block offset** field that selects the word within the block. A multiplexer controlled by the block offset bits selects the requested word from the block. Figure 23 shows the hardware for a cache with $C = 8$ words, $b = 4$ (so $B = 2$ blocks, $S = 2$ sets).

The address field decomposition for this cache:



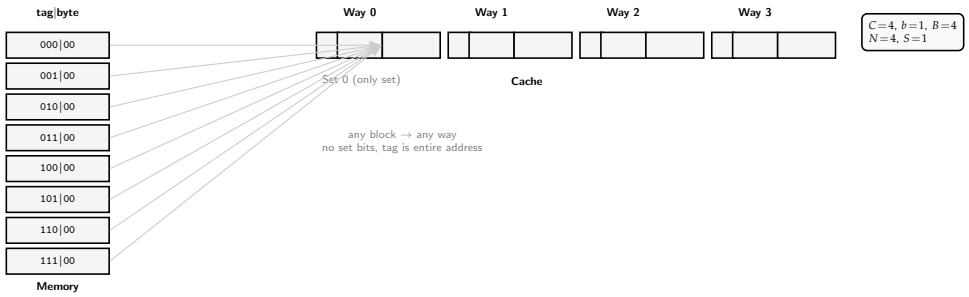


Figure 22: Fully associative cache ($N=4$, $S=1$). No set field — the tag is the entire address (minus byte offset). Any block can go in any way; no conflicts, but requires 4 parallel tag comparators.

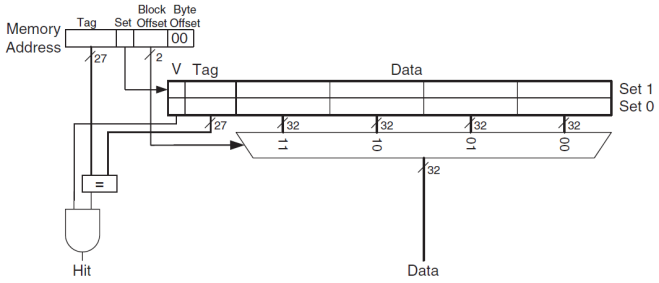


Figure 8.12 Direct mapped cache with two sets and a four-word block size

Figure 23: Direct mapped cache with two sets and a 4-word block size (Harris Figure 8.12). The block offset bits [3:2] control a 4:1 multiplexer to select the word within the block. Only 1 set bit is needed ($\log_2 2 = 1$).

Example: spatial locality (Harris 8.9). Repeating the loop from Example 8.6 with a 4-word block: the first access to address 0x4 misses and loads addresses 0x0–0xC into one cache block. All 14 subsequent accesses (to 0x4, 0xC, 0x8) hit. Miss rate drops from 20% to $1/15 = 6.67\%$.

The trade-off: larger block sizes reduce compulsory misses (spatial locality) but decrease the number of sets in a fixed-size cache, which can increase conflicts. Larger blocks also increase the **miss penalty** — the time to fetch a multi-word block from main memory. For most programs, blocks of 32–64 bytes strike a good balance.

Summary. Table 7 summarises the three cache organisations. In general, the address is decomposed as:

$$\underbrace{\text{Tag}}_{\text{remaining MSBs}} \;\Big|\; \underbrace{\text{Set}}_{\log_2 S \text{ bits}} \;\Big|\; \underbrace{\text{Block Offset}}_{\log_2 b \text{ bits}} \;\Big|\; \underbrace{\text{Byte Offset}}_{\log_2 4{=}2 \text{ bits}}$$

Increasing associativity N reduces conflict misses but requires more comparators. Increasing block size b exploits spatial locality but may increase conflicts and miss penalty.

Organisation	Ways (N)	Sets (S)
Direct Mapped	1	B
N-way Set Associative	$1 < N < B$	B/N
Fully Associative	B	1

Table 7: Cache organisations (Harris Table 8.2). Each address maps to exactly one set but can be stored in any of the N ways.

5.3.3 What Data is Replaced?

In a direct mapped cache, each address maps to a unique block, so replacement is trivial: the old block is evicted. In set associative and fully associative caches, when all ways in a set are full, the cache must choose which block to evict. The principle of temporal locality suggests evicting the **least recently used** (LRU) block, since it is least likely to be needed again soon.

In a 2-way set associative cache, a single **use bit** U tracks which way was least recently used: each time one way is accessed, U is set to indicate the *other* way. On replacement, the block in the way indicated by U is evicted.

For higher associativity ($N > 2$), exact LRU tracking becomes expensive ($\log_2(N!)$ bits per set). In practice, **pseudo-LRU** is used: the ways are divided into two groups, U indicates which group was least recently used, and replacement picks a random block from that group.

5.3.4 Advanced Cache Design

Multi-level caches. Modern systems use at least two cache levels. The **L1 cache** is small and fast (1–2 cycle access); the **L2 cache** is larger but slower (∼10 cycles). The processor checks L1 first; on an L1 miss, it checks L2; on an L2 miss, it goes to main memory.

Reducing miss rate. Cache misses are classified into three types:

- **Compulsory** (cold-start): the first access to a block always misses, regardless of cache design. Larger block sizes reduce compulsory misses (spatial locality).
- **Capacity**: the cache is too small to hold all concurrently needed data. Increasing cache size reduces capacity misses.
- **Conflict**: multiple addresses map to the same set and evict blocks that are still needed. Increasing associativity reduces conflict misses, but beyond 4–8 ways the improvement is small.

Increasing block size reduces compulsory misses but may *increase* conflict misses (fewer sets in a fixed-size cache). For small caches ($\leq$4 KiB), block sizes beyond 64 bytes increase overall miss rate; for larger caches the effect is less pronounced.

Write policy. When the processor stores data to an address that is in the cache, the cache block is updated. The question is whether main memory is also updated:

- **Write-through**: every store simultaneously writes to both the cache and main memory. Simple, but generates many slow main memory writes.
- **Write-back**: the cache block is marked with a **dirty bit** D. $D = 1$ means the block has been modified since it was loaded. Dirty blocks are written back to main memory only when *evicted* from the cache. This is far more efficient: if a block is written k times before eviction, write-back saves $k - 1$ main memory writes.

Modern caches universally use write-back because the main memory access time ($\sim$100 cycles) makes write-through impractical. On a store that misses in the cache, the block is first fetched from main memory into the cache (**write-allocate**), then the store is applied to the cache copy.[10]

[10] **Virtual memory** (Harris §8.4) extends the memory hierarchy to disk: programs use virtual addresses translated to physical addresses via page tables, with a TLB (fully associative cache of page table entries) eliminating the double-access penalty for >99% of accesses. Virtual memory also provides memory protection (per-process page tables). The picorv32 on the iCE40UP5K has no MMU — all addresses are physical — so virtual memory does not apply to this project.

6 Hardware: iCE40UP5K

The iCEBreaker board carries a Lattice **iCE40UP5K** FPGA chip. The board also contains (outside chip) contain a 12 MHz clock, 128 Mbit QSPI flash (W25Q128JV) for bitstream and firmware storage, and an FT2232H for USB programming and UART.

6.1 Resource budget

Table 8 lists the on-chip resources. For the later picorv32 RISC-V implementation, the binding constraints are logic cells (the CPU datapath and control) and memory (instruction/data storage in EBR and SPRAM).

Resource	Count	Notes
Logic Cells (4-LUT + FF)	5280	660 PLBs × 8 LCs; each LC has carry logic
EBR (4 Kb dual-port RAM)	30	120 Kb (15KB) total; configurable as 256×16 to 2048×2
SPRAM (256 Kb single-port)	4	1024 Kb (128KB) total; 16K×16 fixed width, not pre-loadable
DSP (16×16 MAC)	8	≥50 MHz; can split into two 8×8 multipliers
PLL	1	output divider 2^0–2^6; HFOSC 48 MHz / LFOSC 10 kHz also available
I/O pins	39	3 banks: Bank 0 (17), Bank 1 (14), Bank 2 (8)

Table 8: iCE40UP5K-SG48 resource summary.

Figure 24 shows the top-level floorplan. The PLB rows occupy the centre, flanked by columns of EBR (labelled "4 Kb DPRAM") and DSP blocks. The four SPRAM blocks sit along the bottom edge. I/O banks surround the perimeter, with the clock (PLL, HFOSC, LFOSC) along the top.

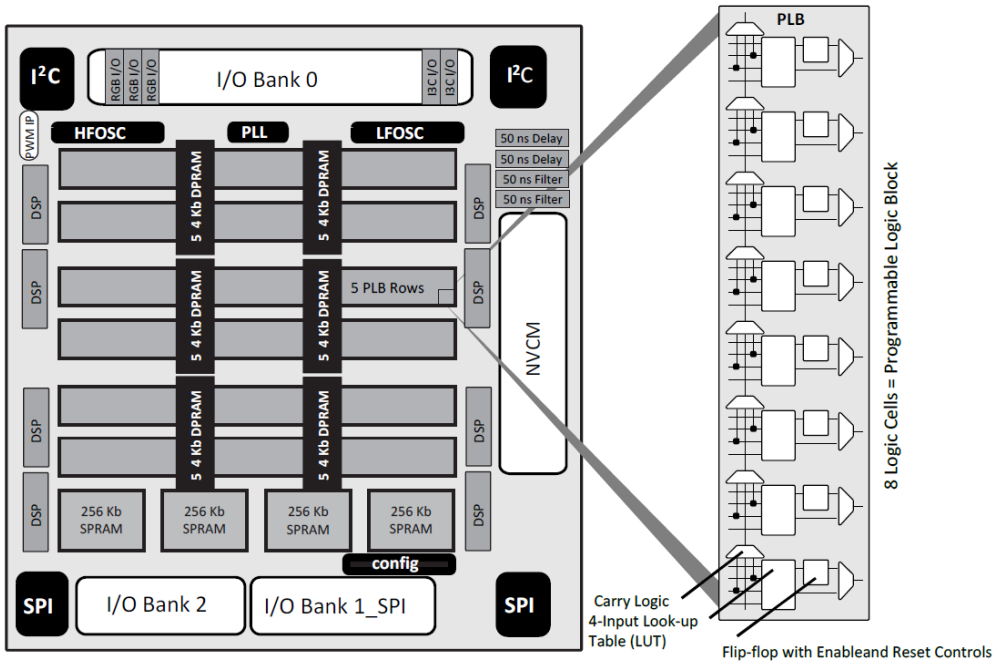


Figure 3.1. iCE40UP5K Device, Top View

Figure 24: iCE40UP5K device floorplan (datasheet Fig. 3.1). The inset shows a single PLB: 8 Logic Cells, each containing a 4-input LUT, carry logic, and a flip-flop.

6.2 Logic Cells and Programmable Logic Blocks

Each **Logic Cell (LC)** pairs one 4-LUT with one D flip-flop and dedicated carry logic:

- **4-LUT**: computes any combinational function of up to 4 inputs. For wider functions (e.g. a 7-input XOR), multiple LUTs are cascaded.
- **D flip-flop**: stores one bit of state, clocked with optional enable and synchronous set/reset. A multiplexer at the output selects between the LUT result (combinational path) and the FF output (registered path).
- **Carry chain**: a dedicated fast-ripple path (FCIN → carry logic → FCOUT) that bypasses the general routing. This lets adders and counters use just one LC per bit — the LUT computes the sum/generate/propagate, and the carry chain handles the carry, without consuming extra LUTs.

Figure 25 shows the detailed structure. Within each LC, the LUT output and the carry logic feed a multiplexer whose output can go directly to the output pin O (combinational) or through the DFF (registered). The carry chain (FCIN/FCOUT) threads vertically through all 8 LCs in the PLB, enabling fast arithmetic without using the general routing fabric.

Eight LCs are grouped into one **Programmable Logic Block (PLB)**, sharing a common clock, clock-enable, and set/reset signal. The shared controls reduce routing overhead — all eight FFs in a PLB are clocked together.

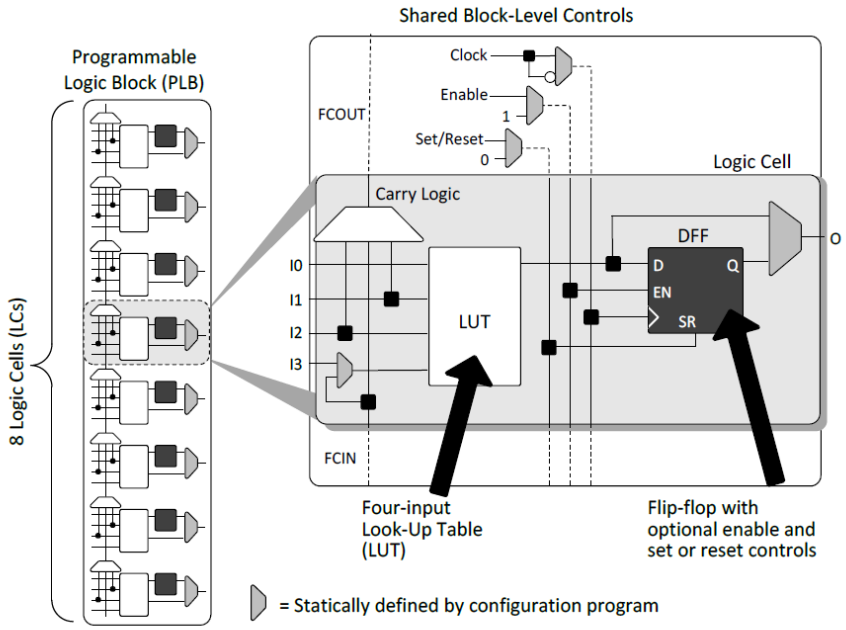


Figure 3.2. PLB Block Diagram

Figure 25: PLB block diagram (datasheet Fig. 3.2). Left: 8 LCs in a column with shared block-level controls (clock, enable, set/reset). Right: detail of one LC showing the 4-input LUT, carry logic, and D flip-flop.

6.3 EBR/DPRAM

The Embedded Block RAM (EBR) blocks are the FPGA's fast, *dual-port* on-chip memory (Dual Port Memory or DPRAM). Each block holds 4 Kbit with a *maximum data-port width of 16 bits*, and can be configured in several aspect ratios:

Configuration	Depth × Width	Address bits
256×16	256 words, 16-bit	8
512×8	512 words, 8-bit	9
1024×4	1024 words, 4-bit	10
2048×2	2048 words, 2-bit	11

Table 9: EBR block configurations (each 4 Kbit).

- **Dual-port (1R1W)**: one read port and one write port, each with its own address, clock, and enable. A reader and a writer can therefore access the block in the same cycle (e.g. the register file reads `rs1` while the previous instruction writes `rd`).
- **Pre-loadable**: contents can be initialised during FPGA configuration (the values are embedded in the bitstream), making EBR usable as ROM or pre-filled RAM. [11]
- **Fast**: operates up to 150 MHz register-to-register.
- **Cascadable**: multiple blocks can be used together for wider or deeper memories. Because the data port is at most 16 bits wide, building a 32-bit width memory must pair two blocks side-by-side (one for bits `[15:0]`, one for `[31:16]`).

6.4 SPRAM

Single Port Memory SPRAM provides bulk storage: four 256 Kbit blocks totalling 1 Mbit (128 KB). Unlike EBR, each SPRAM block is fixed at 16K words × 16 bits with a single read/write port.

- **Single-port**: a given block can either read or write in any clock cycle, never both simultaneously. To build a 32-bit data memory, two SPRAM blocks are used side-by-side (one for the upper 16 bits, one for the lower 16 bits), but both share the same address.
- **Not pre-loadable**: SPRAM contents are undefined after configuration. Firmware must initialise it at runtime.
- **Nibble write masking**: the 4-bit MASKWREN signal allows writing individual nibbles (4-bit groups) within the 16-bit word, enabling byte-granularity stores.
- **Slower**: maximum 70 MHz, compared to EBR's 150 MHz
- **Power**: Supports Standby/Sleep/Power Off modes for reducing power when idle.

6.5 DSP Blocks

The 8 sysDSP blocks are hardened **multiply-accumulate (MAC)** units in dedicated silicon, running at 50 MHz with no LUT cost — implementing the same 16×16 multiply in LUTs would consume hundreds of logic cells. The

[11] However iCEon the iCEBreaker the program lives in SPI flash and is fetched at runtime via `spimemio` (reset vector at `0x00100000`, §7.7). EBR pre-loading is how a BRAM-backed instruction memory *would* work (and how the generic `picosoc_mem` variant loads code), but it is not how this board boots.

picorv32 uses four DSP blocks for `pcpi_fast_mul` (a signed 32×32 multiply built from four 16×16 MACs; Section 7.4), leaving four free.

6.5.1 MAC datapath

Each DSP block performs the **multiply-accumulate (MAC)** operation: acc ← acc + $a \cdot b$. This is the inner loop of matrix multiplication. To compute one output element C_{ij}, the MAC computes the dot product of row $\mathbf{a} = A[i,:]$ and column $\mathbf{b} = B[:,j]$, streaming one element pair per cycle:

$$C_{ij} = \sum_{k=0}^{N-1} A_{ik}\, B_{kj}$$

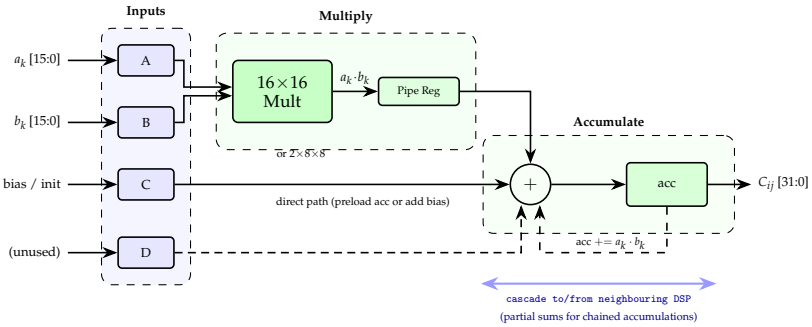
$$= \mathbf{a} \cdot \mathbf{b} = \sum_{k=0}^{N-1} a_k\, b_k$$

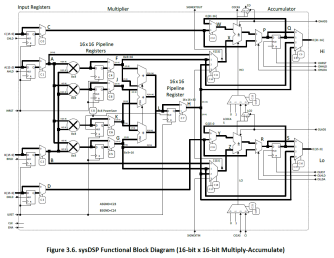
where $\mathbf{a} = A[i,:]$, $\mathbf{b} = B[:,j]$

Algorithm 1 MAC (one DSP, N cycles)

```
acc ← 0                          ▷ preload via C/D
for k = 0 to N − 1 do            ▷ 1 cycle each
    acc ← acc + a_k · b_k
end for
C_ij ← acc
```

For 16-bit integers a_k and b_k, Figure 26 shows the hardware datapath.





Datasheet Fig. 3.6

Figure 26: Left: simplified sysDSP datapath. Inputs a_k, b_k stream through the multiplier; the accumulator feedback (dashed) adds each product to a running sum. C/D bypass the multiplier to preload the accumulator. Cascade signals connect to neighbouring DSPs for chained operations. Right: full datasheet block diagram showing dual 8×8 mode paths.

The multiplier operates in two modes: one 16×16 → 32-bit product, or two independent 8×8 → 16-bit products (for int8 inference). An optional pipeline register between multiplier and accumulator breaks the critical path for higher clock rates at the cost of one cycle latency.

6.5.2 Cascading

Adjacent DSP blocks have dedicated cascade signals (**SIGNEXTIN/SIGNEXTOUT** for sign-extension chains, **CICAS/COCAS** for carry/borrow) that enable wider multiplications or chained accumulations through hardened wiring rather than the general routing fabric. Maximum clock: 50 MHz (with pipeline register bypassed).

6.6 Clocking

The board provides a 12 MHz crystal oscillator. The on-chip PLL synthesises higher frequencies using three divider parameters:

$$f_{\text{out}} = f_{\text{ref}} \times \frac{\texttt{DIVF} + 1}{(\texttt{DIVR} + 1) \times 2^{\texttt{DIVQ}}}$$

where $f_{\text{ref}} = 12$ MHz, `DIVR` is the reference divider (0–15), `DIVF` is the feedback divider (0–63), and `DIVQ` is the output divider (1–6). The VCO frequency must stay within 533–1066 MHz; `DIVQ` then divides the VCO output down to the desired clock. Practically, the Lattice `icepll` tool computes valid settings: `icepll -i 12 -o 24` produces a configuration for 24 MHz.

The maximum frequency for the global clock network is 185 MHz (Table 4.21 of the datasheet). The binding constraints on the system clock for PicoSoC are:

Resource	Max freq	Constraint
Global clock buffer	185 MHz	routing fabric limit
SPRAM	70 MHz	data memory access
DSP (bypassing pipeline reg)	50 MHz	`pcpi_fast_mul`
EBR	150 MHz	instruction cache (if added)

The system clock must not exceed the slowest component on the critical path. With SPRAM at 70 MHz and DSP at 50 MHz, the practical ceiling is ∼48 MHz (allowing margin for routing delay and timing closure). The baseline iCEBreaker design runs at 12 MHz (no PLL); adding a PLL to run at 24 MHz would roughly double throughput with no other changes.

System clock vs. peripheral clocks. The system clock is the single clock that drives the CPU core, EBR, SPRAM, and all on-chip logic. However, external peripherals have their own clocks that are *derived from* the system clock but run at different rates. The most important example is the **SPI flash clock** (SCK): `spimemio` generates the SPI serial clock by dividing the system clock, typically by 2 (so 6 MHz SCK at 12 MHz system). Every SPI transaction is measured in *SPI clocks* — e.g. a QSPI read costs ∼22 SPI clocks — but the CPU stalls for the corresponding number of *system cycles*, which is $22 \times (f_{\text{sys}} / f_{\text{SPI}})$ system cycles. At a 2:1 ratio, 22 SPI clocks = 44 system cycles during which the CPU cannot fetch the next instruction. Raising the system clock with a PLL (with the divider locked at 2:1) raises SCK in lockstep, so a fetch still costs the same *number of system cycles* — only the wall-clock time shrinks with f_{sys}. Cutting the system-cycle cost itself requires either fewer SPI clocks per word (fast-read modes) or a tighter divider ratio. SPRAM and EBR, by contrast, are synchronous to the system clock and respond in 1 system cycle — no clock-domain crossing.

6.7 Power Characteristics

Dynamic power scales linearly with frequency, switching activity, and capacitance ($P_{\text{dyn}} = \alpha C V^2 f$). Static (leakage) power is fixed by the process. The iCE40UP5K datasheet specifies:

Parameter	Typical
Core standby current (V_{CC}=1.2 V, 0 MHz)	75 μA
Core startup peak current	12 mA
SPRAM standby current	0.55 μA

Adding logic (cache controller, pipeline registers, hazard unit) increases both switching activity and capacitive load, raising dynamic power. Enabling the PLL itself draws additional current. However, if the higher clock frequency reduces execution time, the *total energy* per benchmark ($E = P \times t$) may decrease even though instantaneous power rises — the processor finishes sooner and returns to idle.

6.8 Board-Level Connectivity

The iCEBreaker board connects the iCE40UP5K to the rest of the peripherals on board:

- **FT2232H**: USB-to-FPGA bridge providing both JTAG programming (iceprog) and a UART channel (115200 baud). Channel A programs the FPGA; channel B provides serial I/O for the PicoSoC console.
- **12 MHz crystal oscillator**: reference clock for the PLL, shared with the FT2232H.
- **W25Q128JV flash** (128 Mbit): stores the FPGA bitstream and (in PicoSoC) firmware, accessed via QSPI.
- **PMOD headers**: dual PMOD (16 pins) and single PMOD (8 pins). PMOD1A is mapped to a 7-segment display in the modified PicoSoC.
- **LEDs and buttons**: 2 on-board LEDs, 1 push button, plus 5 LEDs and 3 buttons on the snap-off section.

7 picorv32 Microarchitecture

We implement the `picorv32` micoarchitecture on the `icebreaker` chip. Figure 27 shows how the Verilog modules connect: `icebreaker` (top-level board) instantiates `picosoc` system-on-chip plus the board-specific components — four `flash_io_buf` `SB_IO` buffers (the QSPI flash pins) and `seven_seg_ctrl` (which contains `seven_seg_hex`). `picosoc` contains the `picorv32` CPU core, `spimemio` (which contains `spimemio_xfer`, the SPI shift engine), `simpleuart`, and `ice40up5k_spram` (four `SB_SPRAM256KA` SPRAM blocks). The CPU itself contains `picosoc_regs` (register file) and `picorv32_pcpi_fast_mul` (hardware multiplier mapping to DSP blocks).

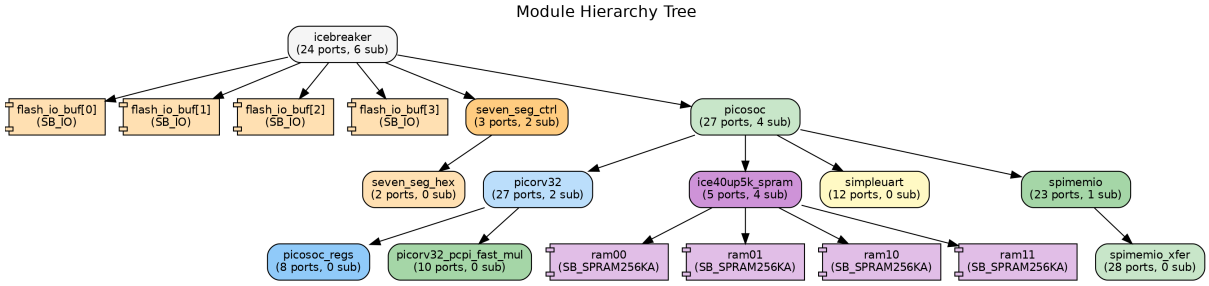


Figure 27: Module hierarchy tree (auto-extracted from yosys elaboration). Each node shows the module name, port count, and number of submodule instances. Leaf nodes with `SB_*` types are iCE40 hard IP primitives.

The picorv32 is highly configurable via Verilog `parameter` declarations. Each parameter trades LUT area against cycle count or critical path length. Table 10 lists the key knobs with their iCEBreaker defaults (from `icebreaker.v`) and trade-off notes.

Parameter	Default	iCEBreaker	Trade-off
`BARREL_SHIFTER`	0	0	**0**: shifts iterate 1–4 bits/cycle in `shift` state (3+N cycles). **1**: combinational barrel shifter in ALU (3 cycles, ∼200 extra LUTs).
`TWO_STAGE_SHIFT`	1	1	When `BARREL_SHIFTER`=0: **1**: shifts 4 bits/cycle when `reg_sh` $\geq$ 4, then 1 bit/cycle. **0**: always 1 bit/cycle (up to 31 cycles).
`TWO_CYCLE_ALU`	0	0	**0**: ALU is combinational (`always @*`). **1**: ALU is registered (`always @(posedge clk)`), adds 1 cycle latency but shortens critical path.
`TWO_CYCLE_COMPARE`	0	0	**1**: branch comparisons take an extra cycle (registers the comparator output). Reduces critical path for branch instructions.
`ENABLE_REGS_DUALPORT`	1	1	**1**: register file has two read ports; both `rs1` and `rs2` read in one cycle (`ld_rs1`). **0**: single read port; R-type/stores need extra `ld_rs2` state (+1 cycle).
`ENABLE_REGS_16_31`	1	1	**0**: only registers `x0`–`x15` (saves LUTs but breaks the ABI — callee-saved `s2`–`s11` unavailable).
`COMPRESSED_ISA`	0	1 → 0	**1**: enables RVC (16-bit instructions). Reduces code size but adds decoder complexity and complicates fetch alignment. **Set to 0** for this project: benchmarks use `-march=rv32im` (no C extension), and disabling RVC simplifies pipelining.
`ENABLE_FAST_MUL`	0	1	**1**: instantiates `pcpi_fast_mul` (uses Verilog `*` → DSP blocks, 2–3 cycles). Mutually exclusive with `ENABLE_MUL`.
`ENABLE_MUL`	0	0	**1**: instantiates `pcpi_mul` (shift-and-add, ∼32 cycles, no DSP).
`ENABLE_DIV`	0	0 → 1	**1**: instantiates `pcpi_div` (restoring division, ∼32-cycle core / ∼40-cycle instruction, ∼250 LUTs). **Must be enabled**: `-march=rv32im` causes gcc to emit `div/rem`; without this, those instructions trap.
`ENABLE_IRQ`	0	1	**1**: enables interrupt support (IRQ FSM, custom instructions, ∼100 LUTs).
`ENABLE_COUNTERS`	1	1	**1**: enables `rdcycle/rdinstret` CSR instructions (64-bit counters).

Table 10: Key picorv32 configuration parameters. The "Default" column is the value in `picorv32.v`; the "iCEBreaker" column is the value set by `icebreaker.v` (via `picosoc.v`), with arrows (→) indicating changes for this project. The modified configuration disables compressed instructions (`COMPRESSED_ISA`=0, not needed for `rv32im`) and enables division (`ENABLE_DIV`=1, required by `rv32im`).

7.1 FSM and Instruction phases

The picorv32 (CPU core) FSM has 8 states, each encoded as a single bit in an 8-bit register (`cpu_state[7:0]`):

Table 11 details each state's micro-operations. Compare with the textbook multicycle state table (Table 4): picorv32 uses only 8 states (vs. 11) by multiplexing — `exec` handles ALU, branches, and JALR; `ld_rs1` dispatches all instruction types. Brackets indicate mux-selected alternatives that depend on the instruction class.

Bit	State	Micro-operation	Description
6	`fetch`	`Instr` ← `Mem[PC]`	Read instruction from memory and latch into IR.
		$\texttt{rd}_{\texttt{prev}}$ ← $\texttt{result}_{\texttt{prev}}$	Deferred writeback of *previous* instruction: `rd` ← [`alu_out_q` | `reg_out` | `PC+2/4`] selected by `latched_stalu/latched_branch`.
		`PC` ← [`PC+2/4` | `target`]	Sequential (`reg_next_pc`) unless previous instruction latched a branch/jump target.[12] Decode Stage 1 fires; speculative prefetch may begin.
5	`ld_rs1`	`reg_op1` ← `cpuregs[rs1]`	Read rs1 from register file. Decode Stage 2 fires (funct3/funct7 → individual `instr_*` flags, compute `decoded_imm`).
		`reg_op2` ← [`rs2` | `imm`]	Read rs2 (R-type, branch, store) or `decoded_imm` (I-type, load). Then dispatch by instruction type. For CSR: `reg_out` ← `counter`, completes immediately. For PCPI: assert `pcpi_valid`, stall until `pcpi_ready`.
4	`ld_rs2`	`reg_op2` ← `cpuregs[rs2]`	Only when `DUALPORT`=0. Read second source register.
		`reg_sh` ← `rs2[4:0]`	Extract shift amount. Then dispatch to `exec`, `stmem`, or `shift`.
3	`exec`	`reg_out` ← `PC + imm`	Unconditionally compute PC-relative target (branch target address).
		`alu_out` ← `op1` *op* `op2`	ALU computes result (add/sub/cmp/logic). For R/I-type ALU: latch `latched_stalu`. For branches: `latched_branch` ← `alu_out_0` (condition result; 0 = not taken). For JALR: latch both `latched_branch` and `latched_stalu` (ALU computes rs1+imm, the register-indirect target).
2	`shift`	`reg_op1` ←/→ `1 or 4`	Shift `reg_op1` by 1 bit (or 4 if `TWO_STAGE_SHIFT` and `reg_sh`≥4).
		`reg_sh` ← `reg_sh` − `1`	Loop until `reg_sh`=0, then `reg_out` ← `reg_op1`.
0	`ldmem`	(1) `reg_op1` ← `op1 + imm`	First visit: compute data address and trigger `mem_do_rdata`.
		(2) `reg_out` ← `Mem[addr]`	Second visit: `mem_done`; capture data (sign/zero-extended per `funct3`).
1	`stmem`	(1) `reg_op1` ← `op1 + imm`	First visit: compute data address and trigger `mem_do_wdata`.
		(2) `Mem[addr]` ← `op2`	Second visit: `mem_done`; data written. No register writeback.
7	`trap`	—	Processor halts; `trap` output asserted. Entered on illegal instruction (if PCPI also rejects), misaligned access, or bus error.

Table 11: picorv32 FSM state table. Each micro-operation has its own row with a matching description. Brackets [a | b] indicate mux-selected alternatives depending on instruction class. Parenthesised (1), (2) indicate first and second visits to the same state.

Instruction paths. Figure 28 shows the state transitions. Every instruction begins with `fetch` → `ld_rs1`, except `jal` which completes entirely within `fetch`. Table 12 lists the path each instruction class takes through the FSM. If `ENABLE_REGS_DUALPORT`=0, R-type, stores, and register shifts insert an extra `ld_rs2` state.

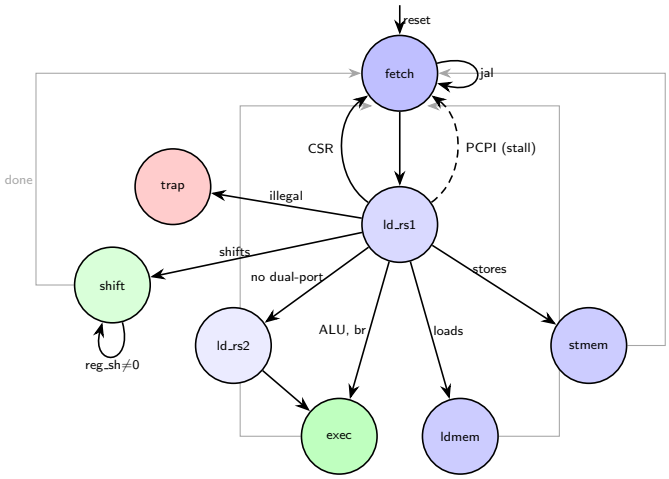


Figure 28: picorv32 FSM. Blue = memory/storage access (fetch, register read, load/store), green = compute (ALU, shift), red = error. Dispatch arrows (black) fan downward from `ld_rs1`. Return arrows (grey) route along the outer edges back to `fetch`, avoiding crossings.

Deviations from textbook. The textbook multicycle FSM (Harris §7.4) has separate Decode and Writeback states. picorv32 eliminates both to save cycles, using three techniques:

[12] The full PC-update mux in `fetch` is: `current_pc = latched_store ? (latched_stalu ? alu_out_q : reg_out) & ~1 : reg_next_pc`. This selects among sequential (PC+2/4), branch target (`reg_out`, PC-relative), or JALR target (`alu_out_q`, register-indirect).

Class	Instructions	Fmt	Path	Cycles
R-type ALU	`add`, `sub`, `and`, `or`, `slt`	R	fetch → ld_rs1 → exec → (fetch)	3
I-type ALU	`addi`, `andi`, `ori`, `slti`	I	fetch → ld_rs1 → exec → (fetch)	3
Upper imm.	`lui`, `auipc`	U	fetch → ld_rs1 → exec → (fetch)	3
Branch (not taken)	`beq`, `bne`, `blt`, ...	B	fetch → ld_rs1 → exec → (fetch)	3
Branch (taken)	`beq`, `bne`, `blt`, ...	B	fetch → ld_rs1 → exec → refetch	5
JALR	`jalr`	I	fetch → ld_rs1 → exec → refetch	6
JAL	`jal`	J	fetch → fetch	3
Load	`lw`, `lh`, `lb`, `lhu`, `lbu`	I	fetch → ld_rs1 → ldmem$^{\times 2}$ → fetch	5
Store	`sw`, `sh`, `sb`	S	fetch → ld_rs1 → stmem$^{\times 2}$ → fetch	5
Shift (no barrel)	`sll`, `srl`, `sra`, ...	R/I	fetch → ld_rs1 → shift$^{\times k}$ → (fetch)	4–14
Shift (barrel)	(same, with `BARREL_SHIFTER`=1)	R/I	fetch → ld_rs1 → exec → (fetch)	3
MUL (fast)	`mul`, `mulh`, `mulhsu`, `mulhu`	R	fetch → ld_rs1 (stall) → (fetch)	∼5
DIV	`div`, `divu`, `rem`, `remu`	R	fetch → ld_rs1 (stall) → (fetch)	∼40
CSR read	`rdcycle`, `rdinstr`	I	fetch → ld_rs1 → (fetch)	2
Illegal	(unrecognised)	—	fetch → ld_rs1 → trap	—

Table 12: picorv32 instruction paths and cycle counts (`ENABLE_REGS_DUALPORT`=1, single-cycle memory). The *Path* column shows the CPU-FSM states traversed; the *Cycles* column is the documented steady-state CPI (picorv32 README), which additionally accounts for the instruction-fetch handshake and prefetch overlap. Every path returns to `fetch`, where the instruction's deferred writeback occurs. A parenthesised `(fetch)` is overlapped with the next instruction via prefetch, so it costs nothing and is not counted; a *plain* trailing `fetch` (loads, stores) is a real cycle the instruction owns, because the data access occupies the shared bus and blocks prefetching — which is why loads and stores cost 5 rather than 3. A *taken* branch or `jalr` likewise cannot use the (wrong-path) prefetch and must *refetch* the real target, costing the extra cycles shown. Superscript $\times k$ = state visited k times (for shifts, k depends on the shift amount with `TWO_STAGE_SHIFT`=1, giving 4–14 cycles). See Table 11 for each state.

1. **Deferred writeback.** There is no Writeback state. Instead, execute/load/store states set *latched* control signals (`latched_store`, `latched_stalu` ("store ALU result"), `latched_branch`, `latched_rd`) recording what to write. The actual register file write happens at the *beginning of the next* `fetch` cycle. This means `fetch` simultaneously writes the previous instruction's result and fetches the next instruction.
2. **Overlapped decode.** Decoding is not a state — it is triggered by signals from the memory interface. When an instruction word arrives (`mem_done`), decoder Stage 1 fires immediately (opcode → group flags, extract rd/rs1/rs2). One cycle later, Stage 2 fires (funct3/funct7 → individual `instr_*` flags, compute `decoded_imm`). Stage 2 completes in the same cycle as `ld_rs1`, so by the time `ld_rs1` dispatches, all instruction flags are valid.
3. **Speculative prefetch.** The memory interface has its own 4-state FSM running in parallel with the CPU FSM. While the CPU is in `ld_rs1` or `exec`, it can speculatively prefetch the next instruction (`mem_do_prefetch`). If the PC advances sequentially, the prefetched instruction is ready immediately; if a branch is taken, the prefetch is discarded.

Walkthrough: `lw t1, 8(s0)` **(5 cycles).** or equivalently `t1 ← Memory[s0 + 8]`. Take the base address which is value of register `s0`, add 8 to it, read `Memory[s0+8]`, then store this value to register `t1`. A single load exercises all three mechanisms above at once — *overlapped decode*, *speculative prefetch*, and *deferred writeback* — and shows why it is one of the most expensive common instructions (only `jalr`/`ret` at 6, and the multi-cycle M-extension, cost more):

Cyc	State	What happens
1	`fetch`	The instruction word arrives. In the *same* cycle the CPU writes back the *previous* instruction's result (deferred writeback). Decoder Stage 1 fires: opcode bits activate flag `is_lb_lh_lw_lbu_lhu`←`1`, extracting `decoded_rs1`←`8` since register s0 is `x8` and `decoded_rd`←`6` since register t1 is `x6`. It also *launches a prefetch* of the next sequential instruction.
2	`ld_rs1`	Decoder Stage 2 resolves `funct3` as `instr_lw`←`1` and the immediate `decoded_imm`←`8`. The value of s0 (which is an address) at base register is read into a working register: `reg_op1 ← cpuregs[8]`.
3	`ldmem`	*First visit.* An adder computes the effective address `reg_op1 ← reg_op1 + decoded_imm` (= s0+8), and `set_mem_do_rdata` issues the *data* read on the shared bus. While this read is in flight, no instruction can be prefetched.
4	`ldmem`	*Second visit.* The data returns (`mem_done`); the aligned word is captured into `reg_out ← mem_rdata_word` (a full word for `lw`; `lh`/`lb` would sign-extend, `lhu`/`lbu` zero-extend). `latched_store ← 1` flags the pending writeback.
5	`fetch`	Back in `fetch`: the deferred writeback completes (`cpuregs[6] ← reg_out`, so t1 now holds the word) *and* the next instruction is fetched and decoded. For an ALU op this fetch would be free — its successor was prefetched during `exec` — but here it is a *real* cycle, because the data access in cycles 3–4 left no bus bandwidth to prefetch.

So the load costs 5 cycles for two compounding reasons: `ldmem` is entered *twice* (cycle 3 computes the address and starts the read; cycle 4 waits for the data), and its trailing `fetch` (cycle 5) is a *real* cycle rather than a free overlap, since the in-flight data access blocked the prefetch. An ALU instruction avoids both: it has a single `exec` step and prefetches its successor, so it finishes in `fetch → ld_rs1 → exec` (3 cycles) with its writeback folded into the next instruction's fetch — the parenthesised `(fetch)` in Table 12.

7.2 Datapath

Figure 29 shows the picorv32 datapath. Like the textbook multicycle processor (Harris Figure 7.27), it consists of state elements connected by nonarchitectural registers and multiplexers that route data differently on each FSM step.

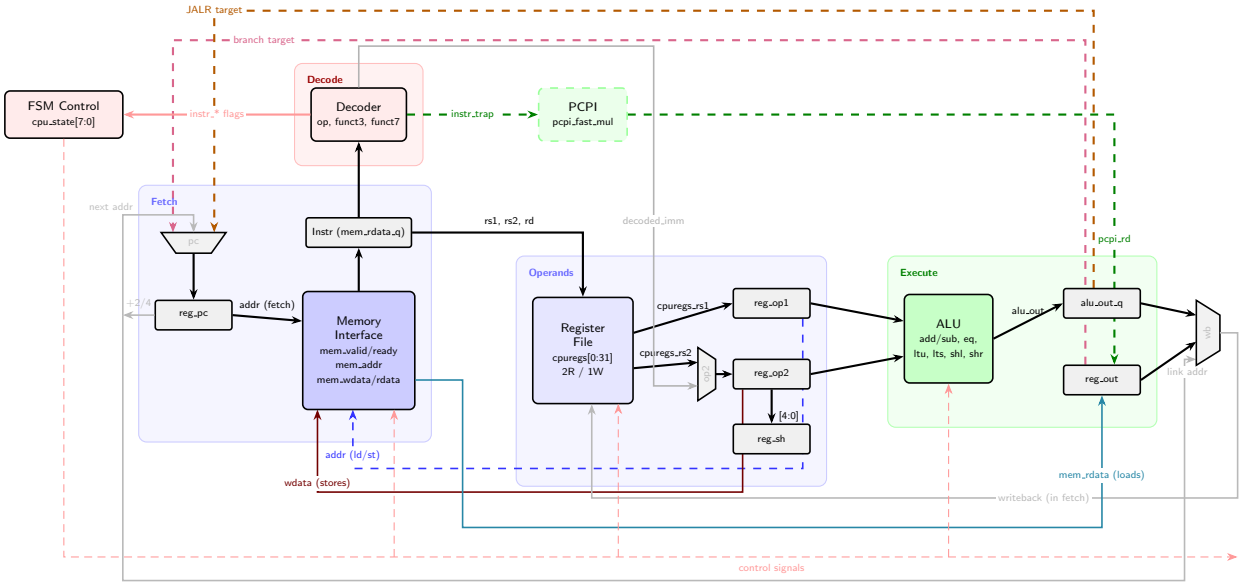


Figure 29: Simplified picorv32 datapath (v2). Three multiplexers control data routing: the **op2 mux** selects the ALU's second operand, the **writeback (wb) mux** selects what is written to the register file, and the **PC mux** selects the next instruction address. A **+2/4 fork** (+2 for `COMPRESSED_ISA` and +4 for full 32bit addresses) from `reg_pc` computes the next sequential address and feeds both the PC mux (as the default "next addr") and the writeback mux (as the "link addr" for JAL/JALR return addresses; note this is a separate adder, not the ALU). **Forward path** (black): PC → Memory Interface → Instruction Register → Decoder (one-hot flags) and Register File (rs1/rs2/rd indices). `reg_op1` receives `cpuregs_rs1`; `reg_op2` receives `cpuregs_rs2` or `decoded_imm` via the op2 mux. Both feed the ALU; its combinational output is registered into `alu_out_q` every cycle. **Feedback paths**: purple dashed = branch target (`reg_out` → PC mux); orange dashed = JALR target (`alu_out_q` → PC mux); teal = load data (`mem_rdata` → `reg_out`); red = store data (`reg_op2` → memory); blue dashed = data address (`reg_op1 + decoded_imm` → memory); green dashed = PCPI (`pcpi_rd` → `reg_out`); grey = +2/4 fork (`reg_pc` → PC mux and writeback mux). Not shown: in the `shift` state, `reg_op1` is shifted in-place (1 or 4 bits/cycle) and `reg_sh` is decremented until zero.

Architectural state. In standalone `picorv32.v` the processor's permanent state is the 32-entry × 32-bit register file and the program counter (`reg_pc`). In standalone `picorv32`, the register file is a Verilog 2D array (`reg [31:0] cpuregs [0:regfile_size-1]`). Each cycle, `decoded_rs1` and `decoded_rs2` holds the address index of the decoded rs1 and rs2 registers. Two combinational read-port wires (`cpuregs_rs1`, `cpuregs_rs2`) hold the value of these addresses, returning 0 when the index is 0 (the x0 convention). Writes are gated by `latched_rd` $\neq$ `0`.

Nonarchitectural working registers. These hold intermediate results between FSM states:

- `reg_op1`: first ALU input (loaded from `cpuregs_rs1` in `ld_rs1`).[13] Repurposed during `ldmem/stmem`: the FSM overwrites it with `reg_op1 + decoded_imm` (i.e. base + offset), so it then holds the computed data memory address that feeds `mem_la_addr`.
- `reg_op2`: second ALU input — either `cpuregs_rs2` (R-type) or `decoded_imm` (I-type). This mux is the key difference between register–register and register–immediate instructions.
- `reg_out`: general result register for non-ALU results: loaded data (`mem_rdata_word` in `ldmem`), shifted value (`reg_op1` in `shift`), PCPI result (`pcpi_int_rd`), CSR values, and branch target (`reg_pc + decoded_imm` in `exec`). *Not* the ALU output — see `alu_out_q` below.
- `alu_out_q`: registered copy of the ALU output, updated every cycle (`alu_out_q <= alu_out`). This is the writeback source for ALU instructions (add, sub, and, or, slt, etc.), selected by `latched_stalu`.
- `reg_sh`: 5-bit shift counter for iterative shifts (decremented each cycle in `shift` state).

At writeback time (beginning of `fetch`), a mux selects the register file write data:

Condition	`cpuregs_wrdata`	Used by
`latched_branch`	`reg_pc + 2/4`	JAL/JALR link address
`latched_stalu`	`alu_out_q`	ALU instructions
otherwise	`reg_out`	Loads, shifts, CSRs, PCPI

ALU. The ALU computes six operations in parallel from `reg_op1` and `reg_op2`. By default these are combinational; with `TWO_CYCLE_ALU`=1 they become registered (adds one cycle latency, shortens critical path):

[13] `cpuregs_rs1` is a combinational wire that reads `cpuregs[decoded_rs1]` — it gives the current *value* stored in the rs1 register. Because it is combinational, it changes instantly whenever `decoded_rs1` changes (i.e. when the next instruction is decoded). `reg_op1` is a clocked flip-flop that *latches* this value so it remains stable across subsequent FSM states. Without the latch, the ALU or memory interface would see the wrong register's contents once the decoder moves on to the next instruction.

Signal	Operation
alu_add_sub	reg_op1 ± reg_op2 (add/sub, also used for address calc and LUI/AUIPC)
alu_eq, alu_lts, alu_ltu	equality, signed less-than, unsigned less-than (for branches and SLT)
alu_shl, alu_shr	left/right shift (used only when BARREL_SHIFTER=1)

An output mux selects the final 32-bit result (alu_out) based on the instruction type: alu_add_sub for arithmetic/address operations, comparison flags for branches/SLT, XOR/OR/AND for logical operations, or shift results when barrel-shifting is enabled. This combinational result is registered into alu_out_q every cycle.

7.3 Instruction decoder

Unlike the textbook decoder (Harris §7.3.3), which produces abstract control signals (RegWrite, ALUSrc, etc.), picorv32 sets **one-hot instruction flags** (instr_add, instr_beq, instr_lw, etc.) that the FSM tests directly. The decode is split across two clock edges:

Stage 1: opcode grouping. Triggered when a new instruction word arrives from memory (mem_do_rinst ("read instruction") && mem_done), Stage 1 examines opcode[6:0] from the raw instruction word and classifies it into one of nine opcode groups. It simultaneously extracts the register indices (decoded_rd from [11:7], decoded_rs1 from [19:15], decoded_rs2 from [24:20]) and pre-computes the J-type immediate decoded_imm_j.

Stage 2: funct3/funct7 refinement. Triggered one cycle later by decoder_trigger, Stage 2 combines the group flags with funct3[14:12] and (for R-type) funct7[31:25] to set individual one-hot instruction signals instr_*. Stage 2 also computes the final decoded_imm via a format-dependent case statement (Section 3, Figure 3).

Table 13 shows the complete two-stage decode: Stage 1 maps the opcode to a group flag (left columns), then Stage 2 refines each group by funct3/funct7 into the final instr_* signal (right columns). LUI, AUIPC, JAL, and JALR are fully resolved in Stage 1 and need no Stage 2 refinement.

Stage 1 (opcode)			Stage 2 (funct3 / funct7)					
opcode	Group flag	Fmt	f3	f7	Signal	Asm	RTL	Description
0110111	instr_lui	U	—	—	instr_lui	lui	rd ← imm ≪ 12	Load upper immediate
0010111	instr_auipc	U	—	—	instr_auipc	auipc	rd ← PC+(imm≪12)	Add upper imm to PC
1101111	instr_jal	J	—	—	instr_jal	jal	rd ← PC+4; PC+=imm	Jump and link
1100111	instr_jalr	I	000	—	instr_jalr	jalr	rd ← PC+4; PC ← rs1+imm	Jump and link register
1100011	is_beq_bne_blt_ bge_bltu_bgeu	B	000	—	instr_beq	beq	if(rs1==rs2) PC+=imm	Branch if equal
			001	—	instr_bne	bne	if(rs1!=rs2) PC+=imm	Branch if not equal
			100	—	instr_blt	blt	if(rs1<rs2) PC+=imm	Branch if < (signed)
			101	—	instr_bge	bge	if(rs1>=rs2) PC+=imm	Branch if ≥ (signed)
			110	—	instr_bltu	bltu	if(rs1<rs2) PC+=imm	Branch if < (unsigned)
			111	—	instr_bgeu	bgeu	if(rs1>=rs2) PC+=imm	Branch if ≥ (unsigned)
0000011	is_lb_lh_lw_ lbu_lhu	I	000	—	instr_lb	lb	rd ← sext(M[8])	Load byte, sign-extend
			001	—	instr_lh	lh	rd ← sext(M[16])	Load half, sign-extend
			010	—	instr_lw	lw	rd ← M[32]	Load word
			100	—	instr_lbu	lbu	rd ← zext(M[8])	Load byte, zero-extend
			101	—	instr_lhu	lhu	rd ← zext(M[16])	Load half, zero-extend
0100011	is_sb_sh_sw	S	000	—	instr_sb	sb	M[8] ← rs2	Store byte
			001	—	instr_sh	sh	M[16] ← rs2	Store halfword
			010	—	instr_sw	sw	M[32] ← rs2	Store word
0010011	is_alu_reg_imm	I	000	—	instr_addi	addi	rd ← rs1+imm	Add immediate
			010	—	instr_slti	slti	rd ← (rs1<imm)?1:0	Set if < imm (signed)
			011	—	instr_sltiu	sltiu	rd ← (rs1<imm)?1:0	Set if < imm (unsign)
			100	—	instr_xori	xori	rd ← rs1^imm	XOR immediate
			110	—	instr_ori	ori	rd ← rs1|imm	OR immediate
			111	—	instr_andi	andi	rd ← rs1&imm	AND immediate
			001	0000000	instr_slli	slli	rd ← rs1≪shamt	Shift left logical imm
			101	0000000	instr_srli	srli	rd ← rs1≫shamt	Shift right logical imm
			101	0100000	instr_srai	srai	rd ← rs1⋙shamt	Shift right arith imm
0110011	is_alu_reg_reg	R	000	0000000	instr_add	add	rd ← rs1+rs2	Add
			000	0100000	instr_sub	sub	rd ← rs1−rs2	Subtract
			001	0000000	instr_sll	sll	rd ← rs1≪rs2	Shift left logical
			010	0000000	instr_slt	slt	rd ← (rs1<rs2)?1:0	Set if < (signed)
			011	0000000	instr_sltu	sltu	rd ← (rs1<rs2)?1:0	Set if < (unsigned)
			100	0000000	instr_xor	xor	rd ← rs1^rs2	XOR
			101	0000000	instr_srl	srl	rd ← rs1≫rs2	Shift right logical
			101	0100000	instr_sra	sra	rd ← rs1⋙rs2	Shift right arithmetic
			110	0000000	instr_or	or	rd ← rs1|rs2	OR
			111	0000000	instr_and	and	rd ← rs1&rs2	AND

Table 13: Unified picorv32 instruction decode table. Stage 1 (left of vertical rule) maps opcode[6:0] to a group flag on the first clock edge; Fmt shows the instruction format (R/I/S/B/U/J from Figure 3). The first four instructions (LUI, AUIPC, JAL, JALR) are fully resolved in Stage 1. Stage 2 (right) refines each remaining group by funct3 (3-bit) and funct7 (7-bit, instr[31:25]) on the next clock edge; "—" means that field is not checked. M[n] denotes an n-bit memory access at address rs1+imm. The M extension (mul/div) shares the R-type opcode 0110011 with funct7=0000001; since the base decoder does not recognise this funct7, instr_trap fires and the instruction is forwarded to PCPI (Table 15, Section 7.4).

Worked example: `add x18, x19, x20`**.** The 32-bit encoding `0x01498933` gives in binary form

$$\underbrace{0000000}_{\text{funct7}}\ \underbrace{10100}_{\text{rs2}=x20}\ \underbrace{10011}_{\text{rs1}=x19}\ \underbrace{000}_{\text{funct3}}\ \underbrace{10010}_{\text{rd}=x18}\ \underbrace{0110011}_{\text{opcode}}$$

Stage 1 sees opcode `0110011` and sets `is_alu_reg_reg` $\leftarrow$ `1`, `decoded_rs1` $\leftarrow$ `19`, `decoded_rs2` $\leftarrow$ `20`, `decoded_rd` $\leftarrow$ `18`. One cycle later, Stage 2 checks `funct3=000` and `funct7=0000000` within the `is_alu_reg_reg` group and sets `instr_add` $\leftarrow$ `1`. The FSM's `ld_rs1` state then sees `instr_add` (via the default branch) and routes to `exec`.

Compressed ISA. When `COMPRESSED_ISA`=1, 16-bit RVC instructions (identified by `instr[1:0]` $\neq$ `2'b11`) are expanded into their 32-bit equivalents during Stage 1 by overwriting fields of `mem_rdata_q`. After expansion, Stage 2 processes them identically to native instructions. RVC reduces code size by 50–60%, which matters when instructions are fetched from slow SPI flash.

The `instr_trap` signal. If none of the recognised `instr_*` flags are set after Stage 2, `instr_trap` is asserted. With `WITH_PCPI`=1, the unrecognised instruction is forwarded to the PCPI coprocessor interface (Section 7.4); if the coprocessor also rejects it, the CPU enters the `trap` state.

7.4 Pico Co-Processor Interface (PCPI)

PCPI is picorv32's native coprocessor extension mechanism. When `instr_trap` fires (unrecognised instruction) and `WITH_PCPI=1`, the instruction is forwarded to external hardware.

Protocol. The PCPI interface is a simple valid/ready handshake with 8 signals:

Signal	Dir	Width	Function
`pcpi_valid`	→	1	CPU asserts when forwarding an unrecognised instruction
`pcpi_insn`	→	32	Full 32-bit instruction word
`pcpi_rs1`	→	32	Source operand 1 (`= reg_op1`)
`pcpi_rs2`	→	32	Source operand 2 (`= reg_op2`)
`pcpi_wait`	←	1	Coprocessor recognises the instruction; stall the CPU
`pcpi_ready`	←	1	Coprocessor is done; result is valid
`pcpi_wr`	←	1	Write result to `rd` (0 = no writeback)
`pcpi_rd`	←	32	Result value

Table 14: PCPI handshake signals. Arrows show direction relative to the CPU (→ = output, ← = input).

The transaction is a short valid/ready handshake (Figure 30). The CPU asserts `pcpi_valid` in `ld_rs1` (or `ld_rs2` without dual-port registers), presenting the instruction word and both operands. A coprocessor that recognises the instruction (from `pcpi_insn[6:0]` and `[14:12]`) asserts `pcpi_wait` to stall the CPU, and on completion asserts `pcpi_ready` with `pcpi_wr=1` and the result on `pcpi_rd`. The CPU captures it into `reg_out`, sets `latched_store=pcpi_wr`, and returns to `fetch`. If nothing responds within ∼16 cycles, `pcpi_timeout` fires and the instruction traps.

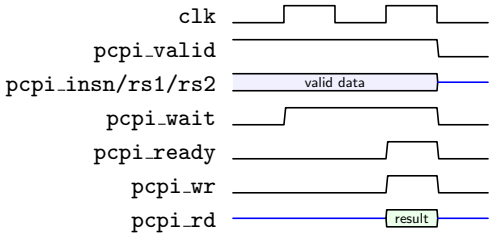


Figure 30: PCPI handshake timing. Cycle 1: CPU asserts `pcpi_valid` with the instruction and operands. Cycle 2: coprocessor recognises the instruction and asserts `pcpi_wait`. Cycles 2–3: computation proceeds. Cycle 4: coprocessor asserts `pcpi_ready` with the result. Cycle 5 (rising edge): CPU captures `pcpi_rd`, deasserts `pcpi_valid`, transitions to `fetch`.

Several coprocessors may be attached at once (Figure 31): inside picorv32 (`picorv32.v` lines 254–345) a priority mux ORs their `wait/ready` lines and forwards the `wr/rd` of whichever asserts `ready` first (external port > multiply > divide). On the iCEBreaker only the two built-in M-extension modules are wired up, so in practice this reduces to "multiply or divide".

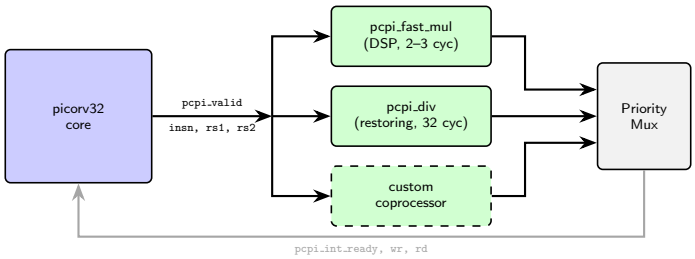


Figure 31: PCPI architecture. The CPU broadcasts `pcpi_valid` with the instruction and operands to all connected coprocessors in parallel. Each coprocessor independently checks whether it handles the instruction. The priority mux combines their responses: the first to assert `ready` wins. On the iCEBreaker only `pcpi_fast_mul` and `pcpi_div` are instantiated — the custom slot (dashed) is unused.

Built-in PCPI modules. picorv32 ships with three optional coprocessors for the M extension: `pcpi_mul` (shift-and-add, ∼32 cycles, pure logic), `pcpi_fast_mul` (uses Verilog `*` operator → maps to iCE40 DSP blocks, 2–3 cycles), and `pcpi_div` (restoring division, ∼32-cycle core / ∼40-cycle instruction). The iCEBreaker rv32im configuration enables *both* `pcpi_fast_mul` (multiply) and `pcpi_div` (divide), since `-march=rv32im` emits the full M-extension.

Table 15 lists the 8 M-extension instructions. All share the R-type opcode `0110011` with `funct7=0000001`. The base decoder (Table 13) does not recognise this funct7, so `instr_trap` fires and the instruction is forwarded via PCPI. The multiply instructions differ only in which half of the 64-bit product they return: `mul` gives the lower 32 bits; `mulh/mulhsu/mulhu` give the upper 32 bits with different sign treatments. A full 64-bit product requires two instructions: `mulh t1, s3, s5` then `mul t2, s3, s5`, giving `{t1,t2} = s3 × s5`.

funct3	funct7	PCPI module	Asm	RTL	Description
000	0000001	pcpi_fast_mul	mul	rd ← (rs1×rs2)[31:0]	Multiply (lower 32 bits)
001	0000001	pcpi_fast_mul	mulh	rd ← (rs1×rs2)[63:32]	Multiply high (signed×signed)
010	0000001	pcpi_fast_mul	mulhsu	rd ← (rs1×rs2)[63:32]	Multiply high (signed×unsigned)
011	0000001	pcpi_fast_mul	mulhu	rd ← (rs1×rs2)[63:32]	Multiply high (unsigned×unsigned)
100	0000001	pcpi_div	div	rd ← rs1 / rs2	Divide (signed)
101	0000001	pcpi_div	divu	rd ← rs1 / rs2	Divide (unsigned)
110	0000001	pcpi_div	rem	rd ← rs1 % rs2	Remainder (signed)
111	0000001	pcpi_div	remu	rd ← rs1 % rs2	Remainder (unsigned)

Table 15: M-extension instructions handled via PCPI. All use R-type format with opcode `0110011` and `funct7=0000001`. On the iCEBreaker, `ENABLE_FAST_MUL`=1 maps multiply to DSP blocks (2–3 cycles) and `ENABLE_DIV`=1 enables hardware division via `pcpi_div` (∼40-cycle instruction, ∼250 LUTs) — required because `-march=rv32im` makes gcc emit `div/rem`.

7.5 Memory Interface

The picorv32 uses a simple valid/ready handshake for **all memory access** — instruction fetches, data loads, and data stores share a single bus. The CPU asserts `mem_valid` with an address and (for writes) data; the memory system responds with `mem_ready` and (for reads) data. A transaction completes on any rising clock edge where both `mem_valid` and `mem_ready` are high.

Three layers. Before the wire-level detail, it helps to see the subsystem as *three layers* (Figure 32)

1. **Internal bus layer** (Table 16). The CPU main FSM (`cpu_state`, Figure 28) never touches the external bus directly. It only raises one-cycle *request* flags — `mem_do_rinst` (fetch an instruction), `mem_do_rdata` (data load), `mem_do_wdata` (data store), `mem_do_prefetch` (speculative fetch) — and later observes `mem_done` (transaction finished) and `mem_rdata_q` (the latched read word). These wires never leave the core.
2. **Bus FSM layer** (Table 17, Figure 34). A small 4-state controller (`mem_state[1:0]`) sitting between the two buses: it consumes those internal requests, runs the valid/ready handshake on the external bus, and reports completion back to the CPU FSM. It is the *only* block that drives the external bus.
3. **External bus layer** (Table 18). The `mem_*` wires (plus the one-cycle-early look-ahead `mem_la_*`) that cross the CPU ↔ memory boundary. PicoSoC's address decoder (Section 7.7) routes them to flash, SPRAM, or a peripheral.

Note that any component → *drives* the next component on the right, which will reply with ← back to left component.

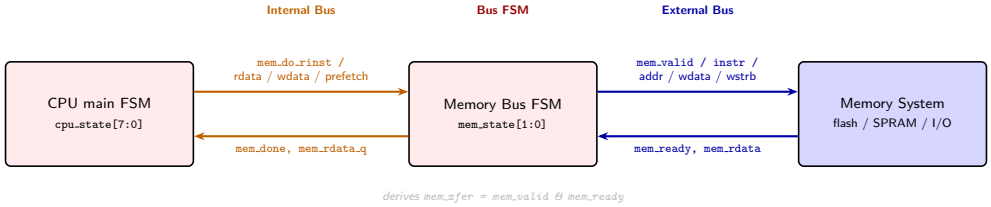


Figure 32: The three layers of the picorv32 memory subsystem (detailed in the text), drawn left-to-right from core to memory: the **internal bus** (orange, the CPU FSM ↔ bus FSM request handshake), the **bus FSM** (red), and the **external bus** (blue, to flash/SPRAM/I/O via PicoSoC's decoder).

7.5.1 Internal bus layer

The CPU main FSM never drives the external bus directly; it talks to the bus FSM over a private handshake (Table 16) that never appears on a pin.

Signal	Dir	Width	Function
`mem_do_rinst`	→	1	CPU FSM requests an instruction fetch
`mem_do_rdata`	→	1	CPU FSM requests a data load
`mem_do_wdata`	→	1	CPU FSM requests a data store
`mem_do_prefetch`	→	1	CPU FSM requests a speculative (next-sequential) fetch
`mem_xfer`	—	1	Derived as `mem_valid & mem_ready`: a bus transfer completes this cycle
`mem_done`	←	1	Bus FSM signals a fetch/load/store completed; releases the decoder (fetches) or `ldmem`/`stmem` (data)
`mem_rdata_q`	←	32	Bus FSM's latched copy of `mem_rdata`, held stable across CPU-FSM states

Table 16: Internal bus signals — the private handshake between the CPU main FSM (`cpu_state`) and the bus FSM (`mem_state`). Direction follows Figure 32: → is CPU FSM → bus FSM, ← is bus FSM → CPU FSM. None of these appear on the external bus (contrast Table 18); `mem_xfer` is not a stored signal but a combinational term used throughout the FSM.

The prefetch allows an extra fetch for free:

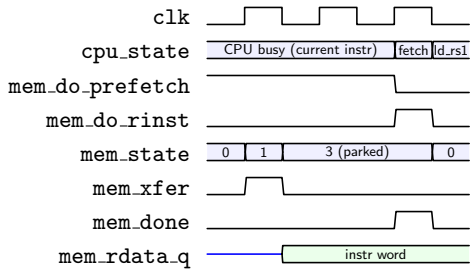


Figure 33: Internal-bus timing for a *prefetched* instruction fetch. While the CPU FSM is busy with a multi-cycle instruction it holds `mem_do_prefetch`, so the bus FSM fetches the *next* instruction in the background: one bus transfer (`mem_xfer`) latches the word into `mem_rdata_q` and the bus FSM *parks* in state 3 *without* firing `mem_done`. The word then waits until the CPU finally reaches `fetch` and raises `mem_do_rinst`: the bus FSM returns state 3 → 0 and fires `mem_done` with *no new bus transaction* — the fetch is free. This is why a prefetch hides flash-fetch latency, and why state 1 routes a prefetch to state 3 rather than state 0 (Table 17 and its footnote).

7.5.2 Bus FSM layer

The bus FSM is a 4-state machine `mem_state[1:0]` that runs in parallel with the CPU FSM: it consumes the internal `mem_do_*` requests and drives the external bus. Table 17 details the states; Figure 34 shows the transitions.

State	Name	Description
0	Idle	No transaction in progress. If `mem_do_rinst`, `mem_do_rdata`, or `mem_do_prefetch` is asserted, asserts `mem_valid` (unless using a prefetched high word) and transitions to state 1. If `mem_do_wdata`, asserts `mem_valid` with write strobes and transitions to state 2.
1	Read pending	Waiting for `mem_ready`. On `mem_xfer` (= `mem_valid & mem_ready`), latches read data into `mem_rdata_q`. If this was an instruction fetch or data read, returns to state 0; if a prefetch, transitions to state 3.[14]
2	Write pending	Waiting for `mem_ready` with write strobes active. On `mem_xfer`, de-asserts `mem_valid` and returns to state 0.
3	Prefetch done	Holds the speculatively fetched instruction. When the CPU FSM later requests an instruction read (`mem_do_rinst`), the prefetched data is already available and the FSM returns to state 0 without issuing a new bus transaction.

Table 17: Memory bus FSM states (`mem_state[1:0]`). The `mem_done` signal fires when a read or write completes (state 1→0 or 2→0), triggering the instruction decoder (for fetches) or allowing `ldmem/stmem` to proceed.

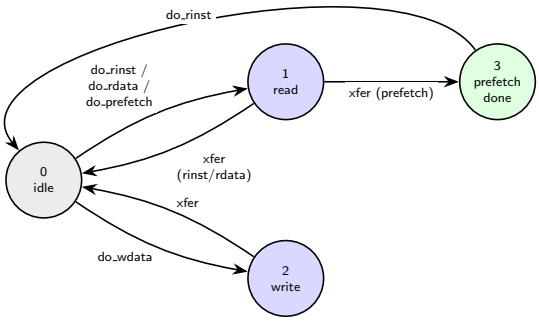


Figure 34: Memory bus FSM. The idle state dispatches to read (state 1) or write (state 2) based on CPU request signals. Reads complete back to idle; prefetches park in state 3 until the CPU needs the instruction. An instruction cache does *not* change this FSM: it sits one layer out as a PicoSoC bus responder (selected by address, §7.7), so the bus FSM still issues `mem_valid` and parks in state 1 — the cache merely returns `mem_ready` in ∼2 cycles on a hit instead of the ∼128 a flash read costs.

7.5.3 External bus layer

Finally, Table 18 lists the wires that actually cross the CPU ↔ memory boundary — the only layer the memory system sees. All CPU outputs are registered; inputs are sampled combinationally. The instruction cache lives at *this* layer: it is one of PicoSoC's external-bus responders (§7.7), answering `mem_ready/mem_rdata` for the flash address range, transparently to the CPU and bus FSM above.

Signal	Dir	Width	Function
`mem_valid`	→	1	CPU asserts to request a transaction
`mem_instr`	→	1	1 = instruction fetch, 0 = data access
`mem_addr`	→	32	Byte address (word-aligned for 32-bit access)
`mem_wdata`	→	32	Write data (valid when `mem_wstrb` ≠ 0)
`mem_wstrb`	→	4	Any 1 bit indicates write byte-enables; `0000` = read
`mem_ready`	←	1	Memory asserts to complete the transaction
`mem_rdata`	←	32	Read data (valid when `mem_ready` & `wstrb`=0)
`mem_la_read`	→	1	Look-ahead: read will be requested next cycle
`mem_la_write`	→	1	Look-ahead: write will be requested next cycle
`mem_la_addr`	→	32	Look-ahead address (available one cycle early)
`mem_la_wdata`	→	32	Look-ahead write data
`mem_la_wstrb`	→	4	Look-ahead write strobes

Table 18: picorv32 external memory-interface signals. The upper group is the primary valid/ready handshake; the lower group is the look-ahead interface that presents each transaction one cycle early. The →/← direction convention is defined in Figure 32 (→ = CPU → memory, ← = memory → CPU).

`mem_instr` lets the memory system route instruction fetches to SPI flash and data accesses to SPRAM. `mem_wstrb` = `0000` signals a read; any non-zero pattern is a write with byte-granularity enables.

Look-ahead interface. Synchronous memories (EBR, SPRAM) need the address one clock cycle before the data is available. The look-ahead signals (`mem_la_*`) provide the address, data, and strobes **one cycle before** `mem_valid`,

[14] The bus FSM does not itself remember the request type; at `mem_xfer` it re-reads the CPU FSM's still-asserted request flags — `mem_state <= mem_do_rinst || mem_do_rdata ? 0 : 3` (`picorv32.v` line 617). Those flags are held for the whole transaction (cleared only by `mem_done`), so `mem_do_prefetch` alone — a speculative fetch the CPU is not yet waiting on — routes to state 3, whereas `mem_do_rinst/mem_do_rdata` (the CPU is stalled now) route back to state 0. Note `mem_instr` cannot make this distinction: it is 1 for both `prefetch` and `rinst`.

so the memory can begin its internal access immediately and have the result ready when `mem_valid` goes high.[15] Figure 35 shows the timing.[16]

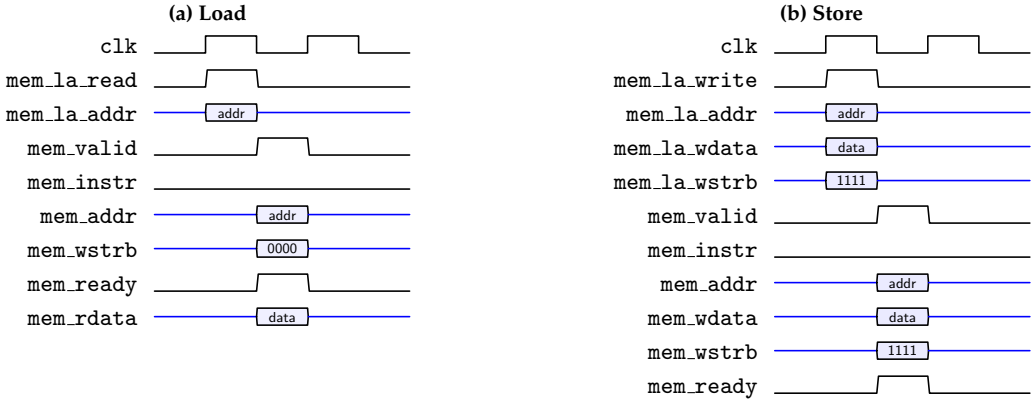


Figure 35: Memory bus timing for (a) a load and (b) a store, drawn with the look-ahead interface. The `mem_la_*` signals present the address (and, for a store, the write data and strobes) **one cycle before** `mem_valid`, giving synchronous memory (EBR/SPRAM) a cycle to register the address. `mem_instr`=0 in both, since these are data accesses (an instruction fetch would assert it); `mem_wstrb`=0000 marks the load as a read, `1111` a full-word store. These are *data* accesses, which in PicoSoC target single-cycle SPRAM; *instruction* fetches instead go to SPI flash and stall for many wait states (Figure 36). Here the memory responds immediately (`mem_ready` in the same cycle as `mem_valid`), so each transaction completes in one `mem_valid` cycle; a slower memory holds `mem_ready` low to insert wait states.

By contrast, an instruction fetch targets SPI flash, and `spimemio` holds `mem_ready` low for many cycles (while `spimemio` is reading).

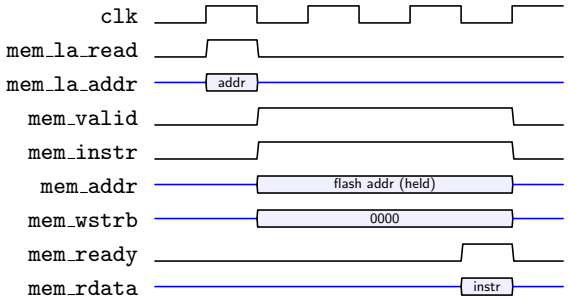


Figure 36: Instruction fetch from SPI flash — the slow path. `mem_instr`=1 routes the request to `spimemio`; `mem_wstrb`=0000 marks it a read. Unlike the single-cycle SPRAM accesses of Figure 35, the flash controller holds `mem_ready` *low* for many SPI cycles, while `mem_valid`/`mem_addr` stay asserted; `mem_rdata` returns only on the final cycle. The wait window here is compressed — a real single-bit fetch is ∼64 SPI clocks ≈ 128 system cycles at the 2:1 SCK divider (§7.6).

[15] PicoSoC does not actually route the look-ahead bus (the `cpu` instance in `picosoc.v` wires only `mem_valid/instr/ready/addr/wdata/wstrb/rdata`); its SPRAM instead uses a registered `ram_ready` pulse, so a data access completes the cycle *after* `mem_valid` (one wait state) rather than the same cycle shown in Figure 35. That same-cycle response is the core interface's look-ahead *capability*, not what this SoC wires up.

[16] How to read the waveforms: a single-bit signal (e.g. `mem_valid`) is one line that toggles high/low. A multi-bit bus (e.g. `mem_addr`, `mem_rdata`) is drawn as a flat line while it carries no meaningful value (idle/don't-care), and *opens into a labelled band* only while it holds a valid value.

7.6 SPI Flash Controller (`spimemio`)

`spimemio` serves instruction fetch requests from the CPU (specifically memory bus FSM) and exposes a config register at `0x02000000`. It has *no cache* — it *streams* sequential reads, advancing word by word. Since all code runs in place from flash (Section 7.7), it sets the fetch latency that dominates the baseline.

Transaction structure. A read is a fixed sequence of phases driven by a 13-state FSM (Table 19, Figure 37); the inner `spimemio_xfer` engine shifts each byte over the flash pins at SCK = half the system clock rate. Single-bit SPI shifts 1 bit per SCK, while Dual-/Quad-I/O shift 2/4 bits — using less number of SPI cycles (Figure 38). These are all *single-data-rate* (SDR): one transfer per rising SCK edge, so the lane count (1/2/4) sets the bits per clock. *Double-data-rate* (DDR) clocks data on *both* the rising and falling edges of SCK, doubling the bits-per-clock again, so quad-DDR (`0xED`) moves 8 bits per SCK, twice quad-SDR. The price is tighter I/O timing, since the controller must drive and sample data on both edges (the negedge-registered `xfer_io*_90` path in `spimemio_xfer`).

State	Phase	Action
0–1	Power-up	Send `0xFF` (terminate any continuous-read mode), then wait.
2–3	Wake	Send `0xAB` (release power-down), then wait.
4	Command	Issue the SPI read opcode — `0x03` (single), `0xBB` (dual), `0xEB` (quad), or `0xED` (quad DDR) — selected by {`config_ddr`, `config_qspi`}.
5–7	Address	Shift 3× byte address (`addr[23:16]`, `[15:8]`, `[7:0]`) for 24 bits
8	Mode + dummy	(QSPI/DDR only) send the continuation byte (`0xA5` if `config_cont`, else `0xFF`) followed by `config_dummy` dummy cycles.
9–12	Data	Read 4 bytes into the 32-bit word. State 12 loops back to 9 for the next sequential word, keeping the command open.

Table 19: `spimemio` read-transaction phases (FSM `state[3:0]`). States 0–3 run once after reset, while states 4–12 form each read. For streaming sequential words, states 9–12 loop over itself.

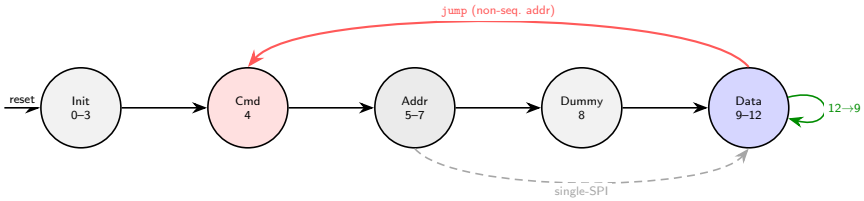


Figure 37: `spimemio` read FSM, grouped by phase (state numbers from Table 19). States 0–3 run once after reset; each read then walks Cmd → Addr → Dummy → Data, and single-bit SPI skips the mode/dummy state (dashed, 7 → 9). Green: sequential streaming — the Data phase loops (12 → 9), advancing `rd_addr` by 4 and keeping the flash command open, so consecutive words cost only their data phase. Red: a `jump` (non-sequential fetch) aborts the burst and re-issues the command and address — the per-branch penalty that a cache or line buffer removes.

Figure 38 gives the per-word cost: ∼64 SPI clocks for single-SPI, ∼20 SCK clocks for Quad-I/O+CRM, while sustained sequential reads take ∼8 SCK clocks. [17]

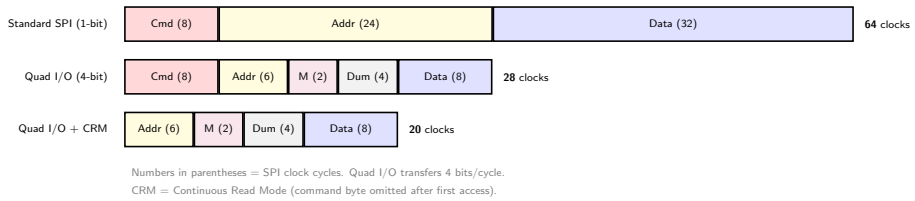


Figure 38: SPI flash read transaction overhead for one 32-bit word. Standard SPI transmits 1 bit per clock: 8 (command) + 24 (address) + 32 (data) = 64 SPI clocks (the `0x03` read has no dummy phase). Quad I/O sends the command single-lane (8 clocks) but transfers address and data 4 bits per clock, reducing the total to ∼28 SPI clocks. CRM omits the command byte on later accesses, cutting this to ∼20. The `spimemio` controller keeps the flash command open and streams sequential words, so sustained throughput in QSPI+CRM drops to ∼8 SPI clocks per word (data phase only).

[17] ∼128, ∼40, and ∼16 *system* cycles at the 2:1 SCK divider

Configuration register. The mode is held in `cfgreg` (Table 20), writable at `0x02000000`. `config_en`=1 lets the controller drive the flash automatically; `config_en`=0 hands the pins to firmware for manual bit-banging — used by `flashio_worker` in `start.s` to program the chip's status registers (e.g. setting the QE bit). The remaining fields select the read command, dummy-cycle count, and continuous-read mode.

Bit(s)	Field	Meaning
`[31]`	`config_en`	MEMIO enable (reset 1; 0 = manual bit-bang)
`[22]`	`config_ddr`	DDR enable (reset 0)
`[21]`	`config_qspi`	Quad-I/O enable (reset 0)
`[20]`	`config_cont`	CRM / continuous-read enable (reset 0)
`[19:16]`	`config_dummy`	dummy cycles after address (reset: 8)
`[11:8]`	`config_oe`	manual output-enables (`config_en`=0 only)
`[5:4]`	`config_csb/clk`	manual CSB / SCK (`config_en`=0 only)
`[3:0]`	`config_do`	manual data-out pins (`config_en`=0 only)

Table 20: `spimemio` `cfgreg` (`0x02000000`), after the upstream picosoc map. Bits `[22:20]` jointly select the read command: {`config_ddr`,`config_qspi`} decodes as `00`→`0x03` single-bit, `10`→`0xBB` Dual I/O (single-rate), `01`→`0xEB` Quad I/O, `11`→`0xED` DDR Quad I/O — so `config_ddr` alone (with `config_qspi`=0) selects *dual*, not double-data-rate. `config_cont` (CRM) then drops the command byte on subsequent reads, sending mode byte `0xA5` instead of `0xFF`. All read-mode bits reset to 0, so the controller powers up in the slowest single-SPI `0x03` mode; `config_dummy` resets to 8 but only applies to the QSPI/DDR commands. The lower fields (`[11:8]`, `[5:4]`, `[3:0]`) are the manual bit-bang interface, active only when `config_en`=0 — used at boot by `flashio_worker` to set the flash chip's QE bit.

Sequential streaming and the jump penalty. A fetch hits immediately when its address matches the streamer's running address; the controller then advances by one word and keeps the flash command open (12 → 9), so consecutive words cost only their data phase. But any *non-sequential* address raises `jump`, aborting the burst and restarting at the command/address phase. Every taken branch, call, and return therefore pays the full command+address+dummy latency — the motivation for a line buffer.

Reset defaults. On reset the controller comes up in its slowest mode — standard single-bit SPI (the `0x03` read, ∼64 SPI clocks/word) — and nothing switches it faster unless firmware writes `cfgreg`. Changing these reset values is the cheapest fetch-latency win.

7.7 PicoSoC

PicoSoC is a minimal system-on-chip wrapper that connects the picorv32 CPU to 3 physical memories and peripherals with no bus fabric — just combinational address decode logic.

7.7.1 Physical Memories

SPRAM (128 KB) Main data memory.[18] SPRAM is hard configured physically to have only 16-bit data width, is *not pre-loadable* and is *single-port* — only one read or write per cycle. Hence to achieve 32-bit width, four `SB_SPRAM256KA` blocks wired as shown in Figure 39: two banks, each composed of two blocks side-by-side (one for `wdata[15:0]`, the other for `wdata[31:16]`), giving 32-bit width per bank. `addr[14]` selects between the two 64 KB banks via `CHIPSELECT`; `addr[13:0]` addresses within a bank.[19]

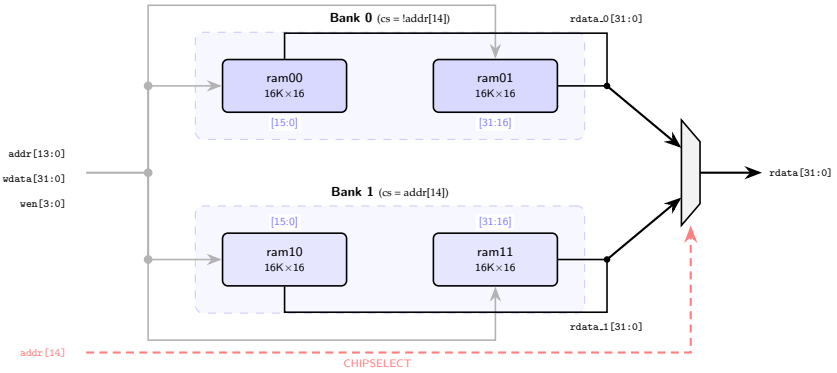


Figure 39: SPRAM wrapper (`ice40up5k_spram.v`). Four `SB_SPRAM256KA` blocks form two banks: within each bank, one block handles the lower 16 bits and the other the upper 16 bits, giving 32-bit width. `addr[14]` selects the bank via `CHIPSELECT`; `addr[13:0]` addresses within a bank (16K words $\times$ 16 bits = 256 Kbit per block). Total: 4 $\times$ 256 Kbit = 128 KB. The single-port constraint means the CPU cannot read and write simultaneously.

SPI flash (16 MB) Instruction storage. The `spimemio` controller fetches instructions over the SPI bus — single-bit by default. It has no instruction cache: it *streams* sequential reads, holding the running address and continuing at `rd_addr+4` without re-sending the command/address, so the first access to a new (non-sequential) address pays the full command + address + dummy penalty while subsequent sequential words are cheap. The flash is selected for `4*MEM_WORDS` (`0x00020000`) $\leq$ SPI addresses $<$ `0x02000000`, so it begins exactly where the SPRAM window ends: a clean partition at `0x00020000`. The reset vector `0x00100000` lies within this flash range.

EBR Register file.[20] The register file (`picosoc_regs`) maps to 4 EBR blocks (Figure 40). The module `picosoc_regs` is a 32$\times$32 array with one 32-bit write port and *two* combinational 32-bit read ports (`rdata1` for `rs1`, `rdata2` for `rs2`). Physically, an EBR block has only one write port and one read port, each at most 16-bit wide. Yosys therefore maps the array to EBR as follows: (i) since the data width maxes at 16 bits, a 32-bit word needs *two* blocks side-by-side (`[15:0]` and `[31:16]`); and (ii) since there is only *one* read port, two simultaneous reads require *duplicating* the whole array into two identical copies (the shared write bus writes both in parallel). Hence 2 copies $\times$ 2 halves = 4 EBR.[21]

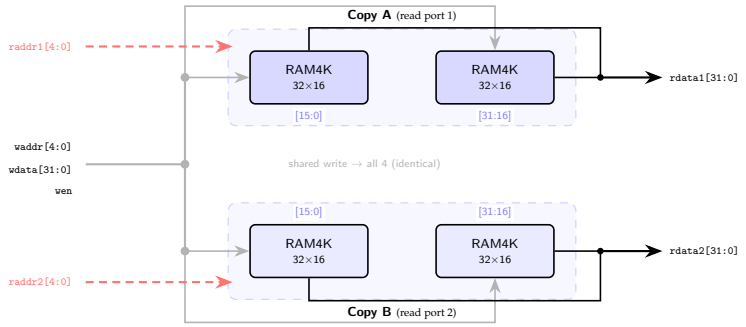


Figure 40: Register file mapped to EBR (*synthesised* mapping — inferred by yosys, not hand-instantiated like the SPRAM). The shared write bus (gray) writes both copies identically; `raddr1`/`raddr2` (red) drive the two independent read ports to rs1 and rs2 register indices respectively, yielding `rdata1`/`rdata2`. The two columns are the 16-bit halves of each 32-bit word; the two rows (Copy A/B) are the duplicate arrays serving the two read ports. Unlike the SPRAM banks (Figure 39), the copies hold *identical* data.

[18] In the default PicoSoC, a small EBR-backed RAM (`picosoc_mem`, 256 words = 1 KB) serves as data memory (stack, locals). However on the iCEBreaker, `icebreaker.v` overrides this with SPRAM (`PICOSOC_MEM = ice40up5k_spram`), so *no EBR is used for main memory*.

[19] `addr` is the wrapper's local port name (`input [21:0] addr`); `picosoc.v` drives it with `mem_addr[23:2]`, of which only `addr[14:0]` carry information — see the address-width note after Table 21.

[20] In the default `picorv32` the register file `cpuregs` synthesises an inferred array of flip-flops; PicoSoC overrides this with `` `define PICORV32_REGS picosoc_regs ``, swapping the internal array for an external module with explicit ports (`waddr`, `raddr1`/`raddr2`, `wdata`, `rdata1`/`rdata2`) that the synthesiser maps to `SB_RAM40_4K` (EBR) block RAM.

[21] Each block is 256 words deep but holds only 32 registers. Setting `ENABLE_REGS_DUALPORT=0` drops the second read port (and one copy), freeing 2 EBR at the cost of an extra `ld_rs2` cycle for two-source instructions.

7.7.2 Address decode

PicoSoC routes each memory access purely by address: a few combinational comparisons on `mem_addr` decide which peripheral responds — range checks for the two memories (SPRAM and flash), exact-address matches for the config and UART registers. The selected device completes the transaction (`mem_ready`) and, on a read, drives its data back to the CPU; if nothing matches, the CPU reads `0`. This is the standard memory-mapped-I/O decoder (Harris §9.2).

Address map. Table 21 shows the full PicoSoC address map as implemented on the iCEBreaker.

Address range	Peripheral	Notes
`0x03000000`	GPIO (in `icebreaker.v`)	`[7:0]`: LEDs. `[15:8]`: 7-segment display via PMOD1A.
`0x02000008`	UART data register	Write: transmit byte. Read: receive byte (-1 if empty).
`0x02000004`	UART baud divisor	`cfg_divider` = f_{clk} / baud $-$ 1.
`0x02000000`	SPI config register	`spimemio cfgreg`: flash mode, latency, DDR/QSPI enable.
`0x00020000–0x01FFFFFF`	SPI flash	Instruction fetch via `spimemio`. Reset vector at `0x00100000`.
`0x00000000–0x0001FFFF`	SPRAM (128 KB)	Data memory (stack, heap, globals). 4×`SB_SPRAM256KA`.

Table 21: PicoSoC address map on the iCEBreaker (higher addresses at top). The SPRAM and flash regions are *contiguous* in the low address space, partitioned at `0x00020000` (= 4×`MEM_WORDS`): addresses below it go to SPRAM, addresses from `0x00020000` to `0x01FFFFFF` go to flash (the reset vector `0x00100000` lies within this range).

Address widths. The CPU always issues a 32-bit `mem_addr`. SPI Flash is byte addressed and only needs `mem_addr[23:0]` (24 bits), because the W25Q128 chip is 16 MB (= 2^{24} bytes) hence spans 24 bits. SPRAM is word-addressed: the wrapper receives `mem_addr[23:2]` — the byte-offset bits `[1:0]` are dropped because each access targets a full 32-bit word, with sub-word writes handled by the `wstrb` byte-enables rather than the address. Of those, only `[14:0]` (15 bits) carry information: 128 KB is 2^{15} = 32768 words, so `addr[13:0]` indexes a bank's 16384 words and `addr[14]` selects the bank (Figure 39). The remaining higher bits are always 0, since the decoder routes only `mem_addr` < `0x00020000` here. Hence 24 byte-address bits for a 16 MB flash, 15 word-address bits for 32768 SPRAM words.

7.7.3 Memory-Mapped I/O

The decode above is **memory-mapped I/O** (Harris §9.2): each peripheral occupies an address range and is accessed with the same `lw`/`sw` instructions as data memory — no special I/O instructions. The textbook hardware pattern (Figure 41) is exactly what PicoSoC builds: an address decoder driving per-device write-enables, and a read multiplexer selecting which device's data returns to the CPU.

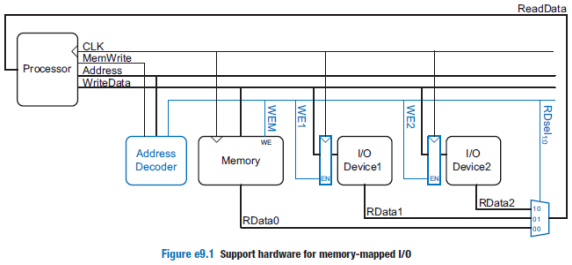


Figure e9.1 Support hardware for memory-mapped I/O

Figure 41: Memory-mapped I/O hardware (Harris Figure e9.1): an address decoder generates per-device write-enables and a read-mux select. PicoSoC realises this pattern — the address comparisons are the decoder, and the read multiplexer returns the addressed device's data.

In firmware, device registers are reached through `volatile` pointers, which compile directly to `lw`/`sw`:

```
#define GPIO_BASE  0x03000000
#define REG_GPIO   (*(volatile uint32_t *)(GPIO_BASE))

REG_GPIO = 0x42;               // sw: set LEDs and 7-segment display
uint32_t val = REG_GPIO;       // lw: read back current GPIO state
```

The `volatile` keyword prevents the compiler from optimising away repeated reads (e.g. in a polling loop).

7.7.4 Boot sequence

On power-up, the picorv32 begins fetching from the reset vector at `PROGADDR_RESET` = `0x00100000` (1 MB into flash). The startup code in `start.s` performs four steps before entering C code:

1. Zero-initialise all 31 general-purpose registers (`x1`–`x31`; `x2`/`sp` is set by the `STACKADDR` parameter to the top of SPRAM).
2. Zero-initialise the entire SPRAM scratchpad (`0x00000000` to `sp`), since SPRAM contents are undefined after configuration.
3. Copy the `.data` section (initialised global variables) from flash to SPRAM, and zero the `.bss` section (uninitialised globals).
4. Call `main()`. On return, an infinite `j loop` halts.

All instructions execute from flash throughout startup (and normal operation).

7.7.5 Module hierarchy

Figure 42 shows the PicoSoC architecture with address decode logic and data return paths. The `picosoc` module instantiates the picorv32 CPU, spimemio (SPI flash controller), simpleuart, and picosoc_mem (EBR or SPRAM). The `icebreaker` module wraps `picosoc` and adds board-specific hardware: the SPRAM wrapper, GPIO register, 7-segment display controller, SB_IO buffers for flash, and the reset counter.

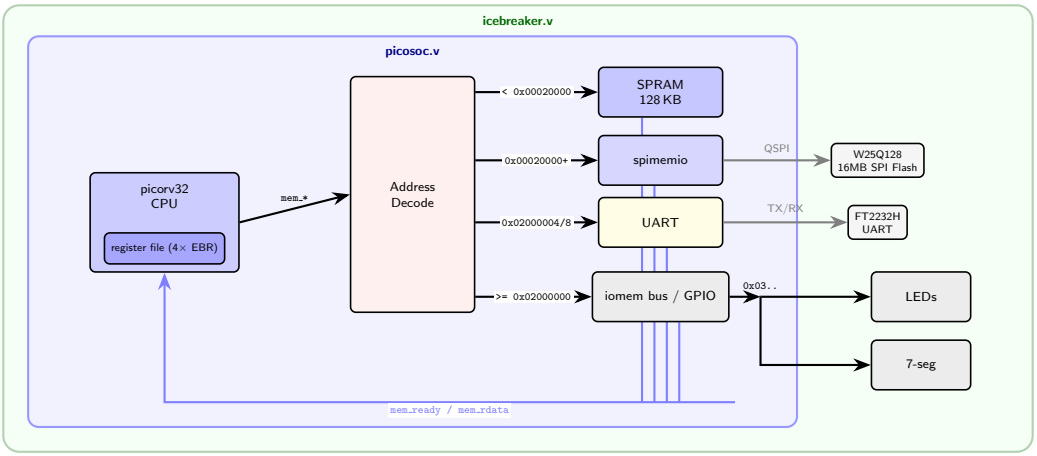


Figure 42: PicoSoC block diagram. Inside `picosoc.v` (blue) the CPU's memory bus fans out by address to SPRAM, SPI flash, and the UART; `icebreaker.v` (green) wraps it and decodes the `iomem` bus (`0x03xxxxxx`) for the LEDs and 7-segment display. The register file is nested inside `picorv32` as 4 EBR blocks — core-internal, so it carries no bus arrow (Figure 40). The SPI config register at `0x02000000` is not a separate peripheral: it is `spimemio`'s `cfgreg` port (flash mode/latency/QSPI). Peripherals return `mem_ready`/`mem_rdata` via the OR/mux chains (blue, bottom).

References

[1] Patterson, D.A. and Hennessy, J.L., *Computer Organization and Design RISC-V Edition: The Hardware Software Interface*, 1st ed., Morgan Kaufmann, 2017.

[2] Harris, S.L. and Harris, D.M., *Digital Design and Computer Architecture*, RISC-V ed., Morgan Kaufmann, 2022.